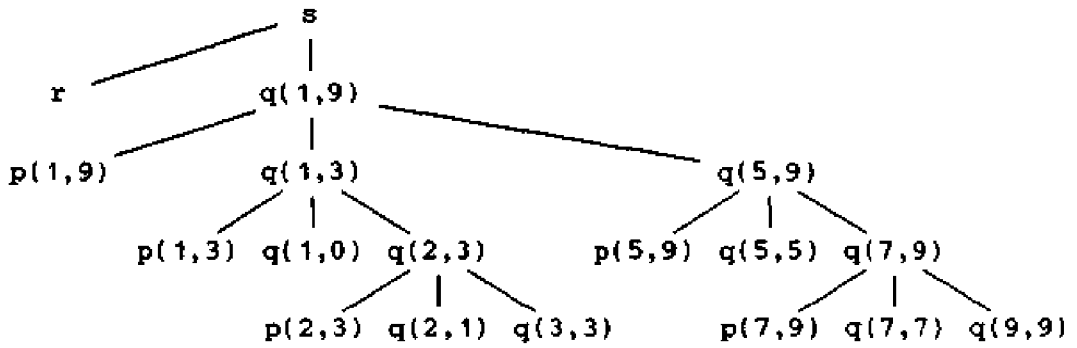


Fig. 7.3. An activation tree corresponding to the output in Fig. 7.2.

Control Stacks

The flow of control in a program corresponds to a depth-first traversal of the activation tree that starts at the root, visits a node before its children, and recursively visits children at each node in a left-to-right order. The output in Fig. 7.2 can therefore be reconstructed by traversing the activation tree in Fig. 7.3, printing `enter` when the node for an activation is reached for the first time and printing `leave` after the entire subtree of the node has been visited during the traversal.

We can use a stack, called a *control stack* to keep track of live procedure activations. The idea is to push the node for an activation onto the control stack as the activation begins and to pop the node when the activation ends. Then the contents of the control stack are related to paths to the root of the activation tree. When node n is at the top of the control stack, the stack contains the nodes along the path from n to the root.

Example 7.2. Figure 7.4 shows nodes from the activation tree of Fig. 7.3 that have been reached when control enters the activation represented by `q(2,3)`. Activations with labels `r`, `p(1,9)`, `p(1,3)`, and `q(1,0)` have executed to completion, so the figure contains dashed lines to their nodes. The solid lines mark the path from `q(2,3)` to the root.

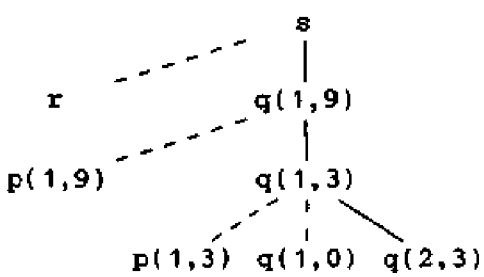


Fig. 7.4. The control stack contains nodes along a path to the root.

At this point the control stack contains the following nodes along this path to the root (the top of the stack is to the right)

```
s, q(1,9), q(1,3), q(2,3)
```

and no other nodes. □

Control stacks extend to the stack storage-allocation technique used to implement languages such as Pascal and C. This technique is discussed in detail in Sections 7.3 and 7.4.

The Scope of a Declaration

A declaration in a language is a syntactic construct that associates information with a name. Declarations may be explicit, as in the Pascal fragment

```
var i : integer;
```

or they may be implicit. For example, any variable name starting with `I` is assumed to denote an integer in a Fortran program, unless otherwise declared.

There may be independent declarations of the same name in different parts of a program. The *scope rules* of a language determine which declaration of a name applies when the name appears in the text of a program. In the Pascal program in Fig. 7.1, `i` is declared thrice, on lines 4, 9, and 13, and the uses of the name `i` in procedures `readarray`, `partition`, and `quicksort` are independent of each other. The declaration on line 4 applies to the uses of `i` on line 6. That is, the two occurrences of `i` on line 6 are in the scope of the declaration on line 4. The three occurrences of `i` on lines 16-18 are in the scope of the declaration of `i` on line 13.

The portion of the program to which a declaration applies is called the *scope* of that declaration. An occurrence of a name in a procedure is said to be *local* to the procedure if it is in the scope of a declaration within the procedure; otherwise, the occurrence is said to be *nonlocal*. The distinction between local and nonlocal names carries over to any syntactic construct that can have declarations within it.

While scope is a property of the declaration of a name, it is sometimes convenient to use the abbreviation "the scope of name `x`" for "the scope of the declaration of name `x` that applies to this occurrence of `x`." In this sense, the scope of `i` on line 17 in Fig. 7.1 is the body of `quicksort`.[2]

At compile time, the symbol table can be used to find the declaration that applies to an occurrence of a name. When a declaration is seen, a symbol-table entry is created for it. As long as we are in the scope of the declaration, its entry is returned when the name in it is looked up. Symbol tables are discussed in Section 7.6.

[2] Most of the time, the terms name, identifier, variable, and lexeme can be used interchangeably without there being any confusion about the intended construct.

Bindings of Names

Even if each name is declared once in a program, the same name may denote different data objects at run time. The informal term "data object" corresponds to a storage location that can hold values.

In programming language semantics, the term *environment* refers to a function that maps a name to a storage location, and the term *state* refers to a function that maps a storage location to the value held there, as in Fig. 7.5. Using the terms *l*-value and *r*-value from Chapter 2, an environment maps a name to an *l*-value, and a state maps the *l*-value to an *r*-value.

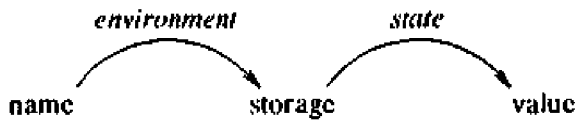


Fig. 7.5. Two-stage mapping from names to values.

Environments and states are different; an assignment changes the state, but not the environment. For example, suppose that storage address 100, associated with variable `pi`, holds `0`. After the assignment `pi := 3.14`, the same storage address is associated with `pi`, but the value held there is `3.14`.

When an environment associates storage location s with a name `x`, we say that `x` is *bound* to s; the association itself is referred to as a *binding* of `x`. The term storage "location" is to be taken figuratively. If `x` is not of a basic type, the storage s for `x` may be a collection of memory words.

A binding is the dynamic counterpart of a declaration, as shown in Fig. 7.6. As we have seen, more than one activation of a recursive procedure can be alive at the same time. In Pascal, a local variable name in a procedure is bound to a different storage location in each activation of a procedure. Techniques for binding local variable names are considered in Section 7.3.

STATIC NOTION	DYNAMIC COUNTERPART
definition of a procedure	activations of the procedure
declaration of a name	bindings of the name
scope of a declaration	lifetime of a binding

Fig. 7.6. Corresponding static and dynamic notions.

Questions to Ask

The way a compiler for a language must organize its storage and bind names is determined largely by answers to questions like the following:

1. May procedures be recursive?
2. What happens to the values of local names when control returns from an activation of a procedure?
3. May a procedure refer to nonlocal names?
4. How are parameters passed when a procedure is called?
5. May procedures be passed as parameters?
6. May procedures be returned as results?
7. May storage be allocated dynamically under program control?
8. Must storage be deallocated explicitly?

The effect of these issues on the run-time support needed for a given programming language is examined in the remainder of this chapter.

7.2 STORAGE ORGANIZATION

The organization of run-time storage in this section can be used for languages such as Fortran, Pascal, and C.

Subdivision of Run-Time Memory

Suppose that the compiler obtains a block of storage from the operating system for the compiled program to run in. From the discussion in the last section, this run-time storage might be subdivided to hold:

1. the generated target code,
2. data objects, and
3. a counterpart of the control stack to keep track of procedure activations.

The size of the generated target code is fixed at compile time, so the compiler can place it in a statically determined area, perhaps in the low end of memory. Similarly, the size of some of the data objects may also be known at compile time, and these too can be placed in a statically determined area, as in Fig. 7.7. One reason for statically allocating as many data objects as possible is that the addresses of these objects can be compiled into the target code. All data objects in Fortran can be allocated statically.

Implementations of languages like Pascal and C use extensions of the control stack to manage activations of procedures. When a call occurs, execution of an activation is interrupted and information about the status of the machine, such as the value of the program counter and machine registers, is saved on the stack. When control returns from the call, this activation can be restarted after restoring the values of relevant registers and setting the program counter to the point immediately after the call. Data objects whose life-

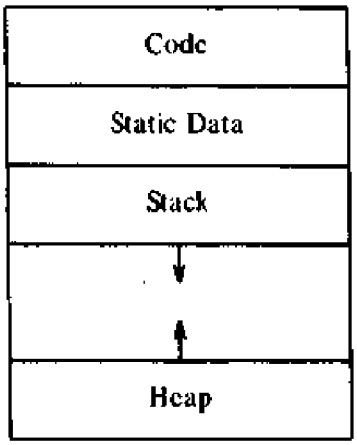


Fig. 7.7. Typical subdivision of run-time memory into code and data areas.

times are contained in that of an activation can be allocated on the stack, along with other information associated with the activation. This strategy is discussed in the next section.

A separate area of run-time memory, called a *heap*, holds all other information. Pascal allows data to be allocated under program control, as discussed in Section 7.7; the storage for such data is taken from the heap. Implementations of languages in which the lifetimes of activations cannot be represented by an activation tree might use the heap to keep information about activations. The controlled way in which data is allocated and deallocated on a stack makes it cheaper to place data on the stack than on the heap.

The sizes of the stack and the heap can change as the program executes, so we show these at opposite ends of memory in Fig. 7.7, where they can grow toward each other as needed. Pascal and C need both a run-time stack and heap, but not all languages do.

By convention, stacks grow down. That is, the "top" of the stack is drawn towards the bottom of the page. Since memory addresses increase as we go down a page, "downwards-growing" means toward higher addresses. If *top* marks the top of the stack, offsets from the top of the stack can be computed by subtracting the offset from *top*. On many machines this computation can be done efficiently by keeping the value of *top* in a register. Stack addresses can then be represented as offsets from *top*.[3]

[3] The organization in Fig. 7.7 assumes that the run-time memory consists of a single contiguous block of storage obtained at the start of execution. This assumption imposes a fixed limit on the combined sizes of the stack and heap. If the limit is large enough that it is rarely exceeded, then it may be wastefully large for most programs. The alternative of linking objects on the stack and heap may make it more expensive to keep track of the top of the stack. Furthermore, the target machine may favor a different placement of areas. For example, some machines allow just positive offsets from an address in a register.

Activation Records

Information needed by a single execution of a procedure is managed using a contiguous block of storage called an *activation record* or *frame*, consisting of the collection of fields shown in Fig. 7.8. Not all languages, nor all compilers use all of these fields; often registers can take the place of one or more of them. For languages like Pascal and C, it is customary to push the activation record of a procedure on the run-time stack when the procedure is called and to pop the activation record off the stack when control returns to the caller.

The purpose of the fields of an activation record is as follows, starting from the field for temporaries.

1. Temporary values, such as those arising in the evaluation of expressions, are stored in the field for temporaries.
2. The field for local data holds data that is local to an execution of a procedure. The layout of this field is discussed below.
3. The field for saved machine status holds information about the state of the machine just before the procedure is called. This information includes the values of the program counter and machine registers that have to be restored when control returns from the procedure.
4. The optional *access link* is used in Section 7.4 to refer to nonlocal data held in other activation records. For a language like Fortran access links are not needed because nonlocal data is kept in a fixed place. Access links, or the related "display" mechanism, are needed for Pascal.
5. The optional *control link* points to the activation record of the caller.

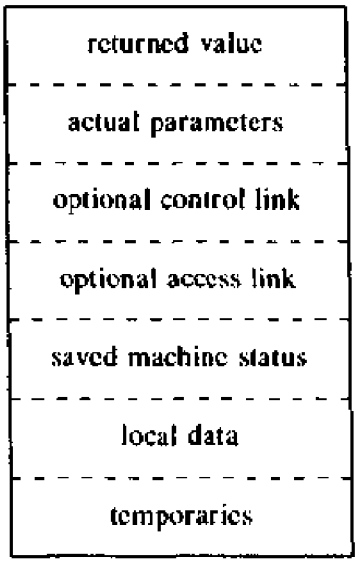


Fig. 7.8. A general activation record.

6. The field for actual parameters is used by the calling procedure to supply parameters to the called procedure. We show space for parameters in the activation record, but in practice parameters are often passed in machine registers for greater efficiency.

7. The field for the returned value is used by the called procedure to return a value to the calling procedure. Again, in practice this value is often returned in a register for greater efficiency.

The sizes of each of these fields can be determined at the time a procedure is called. In fact, the sizes of almost all fields can be determined at compile time. An exception occurs if a procedure may have a local array whose size is determined by the value of an actual parameter, available only when the procedure is called at run time. See Section 7.3 for the allocation of variable-length data in an activation record.

Compile-Time Layout of Local Data

Suppose run-time storage comes in blocks of contiguous bytes, where a byte is the smallest unit of addressable memory. On many machines, a byte is eight bits and some number of bytes form a machine word. Multibyte objects are stored in consecutive bytes and given the address of the first byte.

The amount of storage needed for a name is determined from its type. An elementary data type, such as a character, integer, or real, can usually be stored in an integral number of bytes. Storage for an aggregate, such as an array or record, must be large enough to hold all its components. For easy access to the components, storage for aggregates is typically allocated in one contiguous block of bytes. See Sections 8.2 and 8.3 for more details.

The field for local data is laid out as the declarations in a procedure are examined at compile time. Variable-length data is kept outside this field. We keep a count of the memory locations that have been allocated for previous declarations. From the count we determine a *relative* address of the storage for a local with respect to some position such as the beginning of the activation record. The relative address, or *offset*, is the difference between the addresses of the position and the data object.

The storage layout for data objects is strongly influenced by the addressing constraints of the target machine. For example, instructions to add integers may expect integers to be *aligned*, that is, placed at certain positions in memory such as an address divisible by 4. Although an array of ten characters needs only enough bytes to hold ten characters, a compiler may therefore allocate 12 bytes, leaving 2 bytes unused. Space left unused due to alignment considerations is referred to as *padding*. When space is at a premium, a compiler may *pack* data so that no padding is left; additional instructions may then need to be executed at run time to position packed data so that it can be operated on as if it were properly aligned.

Example 7.3. Figure 7.9 is a simplification of the data layout used by C compilers for two machines that we call Machine 1 and Machine 2. C provides for three sizes of integers, declared using the keywords `short`, `int`, and `long`. The instruction sets of the two machines are such that the compiler for Machine 1 allocates 16, 32, and 32 bits for the three sizes of integers, while the compiler for Machine 2 allocates 24, 48, and 64 bits, respectively. For comparison between machines, sizes are measured in bits in Fig. 7.9, even though neither machine allows bits to be addressed directly.

The memory of Machine 1 is organized into bytes consisting of 8 bits each. Even though every byte has an address, the instruction set favors short integers being positioned at bytes whose addresses are even, and integers being positioned at addresses that are divisible by 4. The compiler places short integers at even addresses, even if it has to skip a byte as padding in the process. Thus, four bytes, consisting of 32 bits, may be allocated for a character followed by a short integer.

In Machine 2, each word consists of 64 bits, and 24 bits are allowed for the address of a word. There are 64 possibilities for the individual bits inside a word, so 6 additional bits are needed to distinguish between them. By design, a pointer to a character on Machine 2 takes 30 bits — 24 to find the word and 6 for the position of the character inside the word.

The strong word orientation of the instruction set of Machine 2 has led the compiler to allocate a complete word at a time, even when fewer bits would suffice to represent all possible values of that type; e.g., only 8 bits are needed to represent a character. Hence, under alignment, Fig. 7.9 shows 64 bits for each type. Within each word, the bits for each basic type are in specified positions. Two words consisting of 128 bits would be allocated for a character followed by a short integer, with the character using only 8 of the bits in the first word and the short integer using only 24 of the bits in the second word. □

TYPE	SIZE (Bits)		ALIGNMENT (Bits)	
	Machine 1	Machine 2	Machine 1	Machine 2
`char`	8	8	8	64[a]
`short`	16	24	16	64
`int`	32	48	32	64
`long`	32	64	32	64
`float`	32	64	32	64
`double`	64	128	32	64
character pointer	32	30	32	64
other pointers	32	24	32	64
structures	≥8	≥64	32	64

[a] Characters in an array are aligned every 8 bits.

Fig. 7.9. Data layouts used by two C compilers.

7.3 STORAGE-ALLOCATION STRATEGIES

A different storage-allocation strategy is used in each of the three data areas in the organization of Fig. 7.7.

1. Static allocation lays out storage for all data objects at compile time.
2. Stack allocation manages the run-time storage as a stack.
3. Heap allocation allocates and deallocates storage as needed at run time from a data area known as a heap.

These allocation strategies are applied in this section to activation records. We also describe how the target code of a procedure accesses the storage bound to a local name.

Static Allocation

In static allocation, names are bound to storage as the program is compiled, so there is no need for a run-time support package. Since the bindings do not change at run time, every time a procedure is activated, its names are bound to the same storage locations. This property allows the values of local names to be *retained* across activations of a procedure. That is, when control returns to a procedure, the values of the locals are the same as they were when control left the last time.

From the type of a name, the compiler determines the amount of storage to set aside for that name, as discussed in Section 7.2. The address of this storage consists of an offset from an end of the activation record for the procedure. The compiler must eventually decide where the activation records go, relative to the target code and to one another. Once this decision is made, the position of each activation record, and hence of the storage for each name in the record is fixed. At compile time we can therefore fill in the addresses at which the target code can find the data it operates on. Similarly, the addresses at which information is to be saved when a procedure call occurs are also known at compile time.

However, some limitations go along with using static allocation alone.

1. The size of a data object and constraints on its position in memory must be known at compile time.
2. Recursive procedures are restricted, because all activations of a procedure use the same bindings for local names.
3. Data structures cannot be created dynamically, since there is no mechanism for storage allocation at run time.

Fortran was designed to permit static storage allocation. A Fortran program consists of a main program, subroutines, and functions (call them all *procedures*), as in the Fortran 77 program of Fig. 7.10. Using the memory organization of Fig. 7.7, the layout of the code and the activation records for

```
(1)      PROGRAM CNSUME
(2)         CHARACTER * 50 BUF
(3)         INTEGER NEXT
(4)         CHARACTER C, PRDUCE
(5)         DATA NEXT /1/, BUF /' '/
(6)  6         C = PRDUCE()
(7)            BUF(NEXT:NEXT) = C
(8)            NEXT = NEXT + 1
(9)            IF ( C .NE. ' ' ) GOTO 6
(10)           WRITE (*,'(A)') BUF
(11)           END

(12)     CHARACTER FUNCTION PRDUCE()
(13)        CHARACTER * 80 BUFFER
(14)        INTEGER NEXT
(15)        SAVE BUFFER, NEXT
(16)        DATA NEXT /81/
(17)           IF ( NEXT .GT. 80 ) THEN
(18)              READ (*,'(A)') BUFFER
(19)              NEXT = 1
(20)           END IF
(21)           PRDUCE = BUFFER(NEXT:NEXT)
(22)           NEXT = NEXT+1
(23)           END
```

Fig. 7.10. A Fortran 77 program.

this program is shown in Fig. 7.11. Within the activation record for CNSUME (read "consume" — Fortran does not like long identifiers), there is space for the locals BUF, NEXT, and C. The storage bound to BUF holds a string of fifty characters. It is followed by space for holding an integer value for NEXT and a character value for C. The fact that NEXT is also declared in PRDUCE presents no problem, because the locals of the two procedures get space in their respective activation records.

Since the sizes of the executable code and the activation records are known at compile time, memory organizations other than the one in Fig. 7.11 are possible. A Fortran compiler might place the activation record for a procedure together with the code for that procedure. On some computer systems, it is feasible to leave the relative position of the activation records unspecified and allow the link editor to link activation records and executable code.

Example 7.4. The program in Fig. 7.10 relies on the values of locals being retained across procedure activations. A SAVE statement in Fortran 77 specifies that the value of a local at the beginning of an activation must be the same as that at the end of the last activation. Initial values for these locals can be specified using a DATA statement.

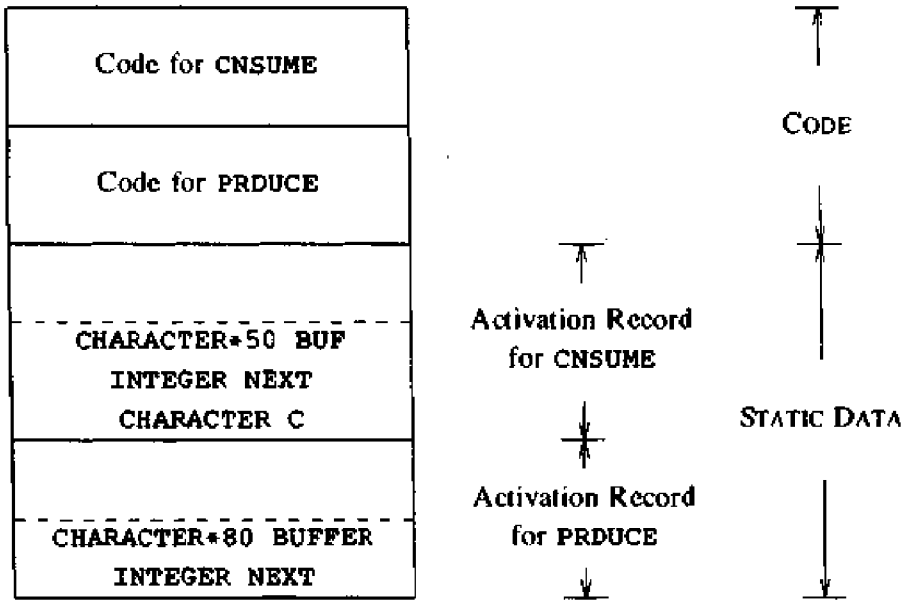


Fig. 7.11. Static storage for local identifiers of a Fortran 77 program.

The statement on line 18 of procedure PRDUCE reads one line of text at a time into a buffer. The procedure delivers successive characters each time it is activated. The main program CNSUME also has a buffer in which it accumulates characters until a blank is seen. On input

```
hello world
```

the characters returned by the activations of PRDUCE are depicted in Fig. 7.12; the output of the program is

```
hello
```

The buffer into which PRDUCE reads lines must retain its value between activations. The SAVE statement on line 15 ensures that when control returns to PRDUCE, locals BUFFER and NEXT have the same values as they did when control left the procedure the last time. The first time control reaches PRDUCE, the value of the local NEXT is taken from the DATA statement on line 16. In this way, NEXT is initialized to 81. □

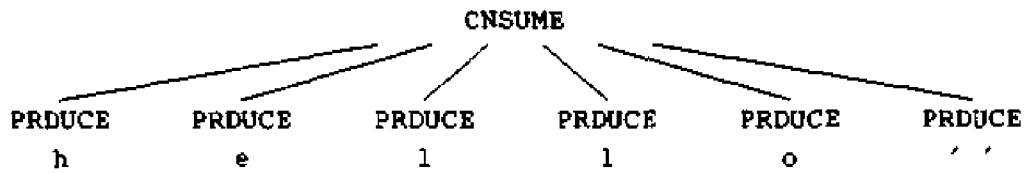


Fig. 7.12. Characters returned by activations of PRDUCE.

Stack Allocation

Stack allocation is based on the idea of a control stack; storage is organized as a stack, and activation records are pushed and popped as activations begin and end, respectively. Storage for the locals in each call of a procedure is contained in the activation record for that call. Thus locals are bound to fresh storage in each activation, because a new activation record is pushed onto the stack when a call is made. Furthermore, the values of locals are *deleted* when the activation ends; that is, the values are lost because the storage for locals disappears when the activation record is popped.

We first describe a form of stack allocation in which the sizes of all activation records are known at compile time. Situations in which incomplete information about sizes is available at compile time are considered below.

Suppose that register *top* marks the top of the stack. At run time, an activation record can be allocated and deallocated by incrementing and decrementing *top*, respectively, by the size of the record. If procedure q has an activation record of size a, then *top* is incremented by a just before the target code of q is executed. When control returns from q, *top* is decremented by a.

Example 7.5. Figure 7.13 shows the activation records that are pushed onto and popped from the run-time stack as control flows through the activation tree of Fig. 7.3. Dashed lines in the tree go to activations that have ended. Execution begins with an activation of procedure s. When control reaches the first call in the body of s, procedure r is activated and its activation record is pushed onto the stack. When control returns from this activation, the record is popped leaving just the record for s in the stack. In the activation of s, control then reaches a call of q with actuals 1 and 9, and an activation record is allocated on top of the stack for an activation of q. Whenever control is in an activation, its activation record is at the top of the stack.

Several activations occur between the last two snapshots in Fig. 7.13. In the last snapshot in Fig. 7.13, activations p(1,3) and q(1,0) have begun and ended during the lifetime of q(1,3), so their activation records have come and gone from the stack, leaving the activation record for q(1,3) on top. □

In a Pascal procedure, we can determine a relative address for local data in an activation record, as discussed in Section 7.2. At run time, suppose *top* marks the location of the end of a record. The address of a local name x in the target code for the procedure might therefore be written as $dx(top)$, to indicate that the data bound to x can be found at the location obtained by adding dx to the value in register *top*. Note that addresses can alternatively be taken as offsets from the value in any other register r pointing to a fixed position in the activation record.

Calling Sequences

Procedure calls are implemented by generating what are known as *calling sequences* in the target code. A *call sequence* allocates an activation record

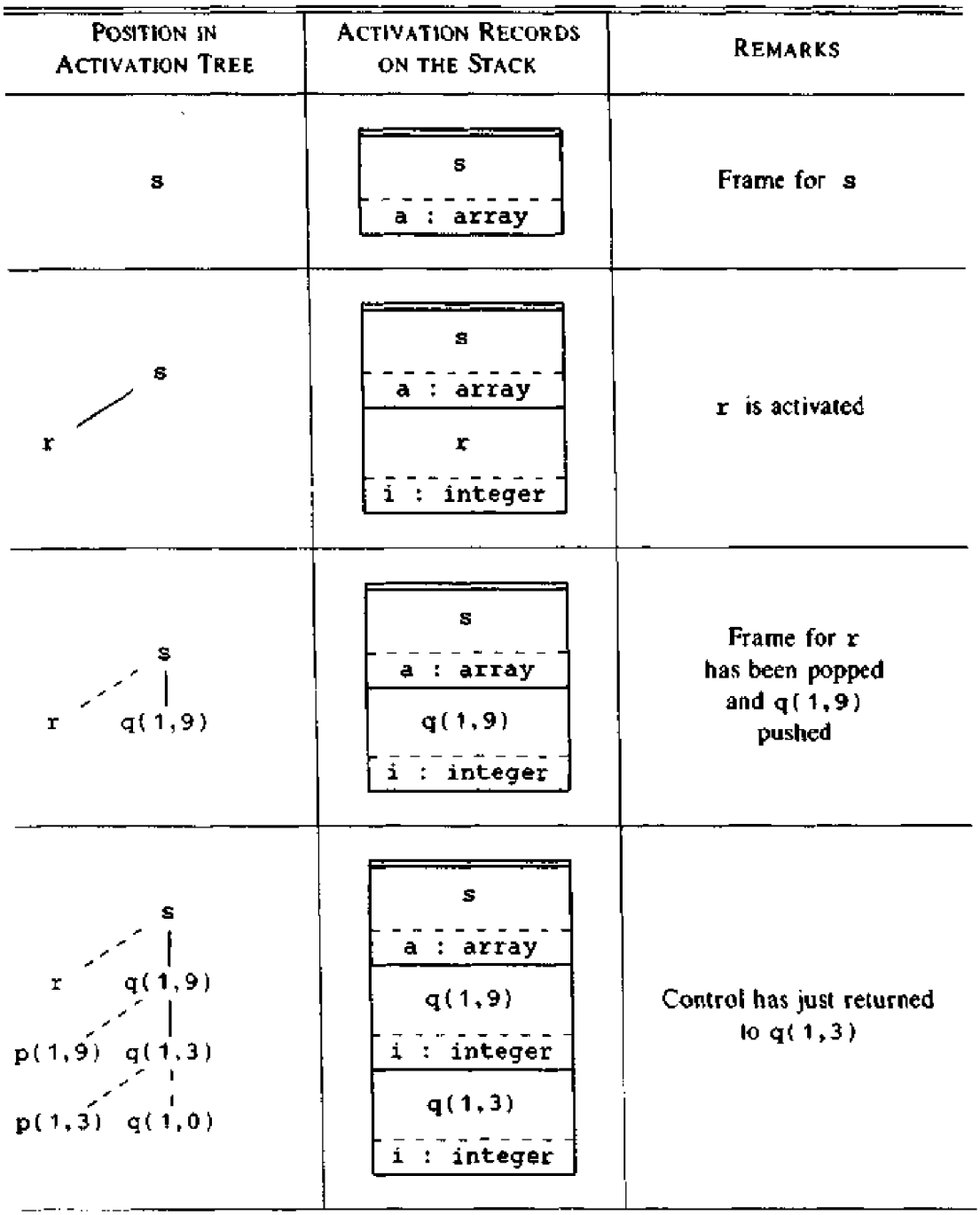


Fig. 7.13. Downward-growing stack allocation of activation records.

and enters information into its fields. A *return sequence* restores the state of the machine so the calling procedure can continue execution.

Calling sequences and activation records differ, even for implementations of the same language. The code in a calling sequence is often divided between the calling procedure (the caller) and the procedure it calls (the callee). There is no exact division of run-time tasks between the caller and callee — the

source language, the target machine, and the operating system impose requirements that may favor one solution over another.[4]

A principle that aids the design of calling sequences and activation records is that fields whose sizes are fixed early are placed in the middle. In the general activation record of Fig. 7.8, the control link, access link, and machine-status fields appear in the middle. The decision about whether or not to use control and access links is part of the design of the compiler, so these fields can be fixed at compiler-construction time. If exactly the same amount of machine-status information is saved for each activation, then the same code can do the saving and restoring for all activations. Moreover, programs such as debuggers will have an easier time deciphering the stack contents when an error occurs.

Even though the size of the field for temporaries is eventually fixed at compile time, this size may not be known to the front end. Careful code generation or optimization may reduce the number of temporaries needed by the procedure, so as far as the front end is concerned, the size of this field is unknown. In the general activation record, we therefore show this field after that for local data, where changes in its size will not affect the offsets of data objects relative to the fields in the middle.

Since each call has its own actual parameters, the caller usually evaluates actual parameters and communicates them to the activation record of the callee. Methods for passing parameters are discussed in Section 7.5. In the run-time stack, the activation record of the caller is just below that for the callee, as in Fig. 7.14. There is an advantage to placing the fields for parameters and a potential returned value next to the activation record of the caller. The caller can then access these fields using offsets from the end of its own activation record, without knowing the complete layout of the record for the callee. In particular, there is no reason for the caller to know about the local data or temporaries of the callee. A benefit of this information hiding is that procedures with variable numbers of arguments, such as `printf` in C, can be handled, as discussed below.

Languages like Pascal require arrays local to a procedure to have a length that can be determined at compile time. More often, the size of a local array may depend on the value of a parameter passed to the procedure. In that case, the size of all the data local to the procedure cannot be determined until the procedure is called. Techniques for handling variable-length data are discussed later in this section.

The following call sequence is motivated by the above discussion. As in Fig. 7.14, the register *top_sp* points to the end of the machine-status field in an activation record. This position is known to the caller, so it can be made responsible for setting *top_sp* before control flows to the called procedure.

[4] If a procedure is called n times, then the portion of the calling sequence in the various callers is generated n times. However, the portion in the callee is shared by all calls, so it is generated just once. Hence, it is desirable to put as much of the calling sequence into the callee as possible.

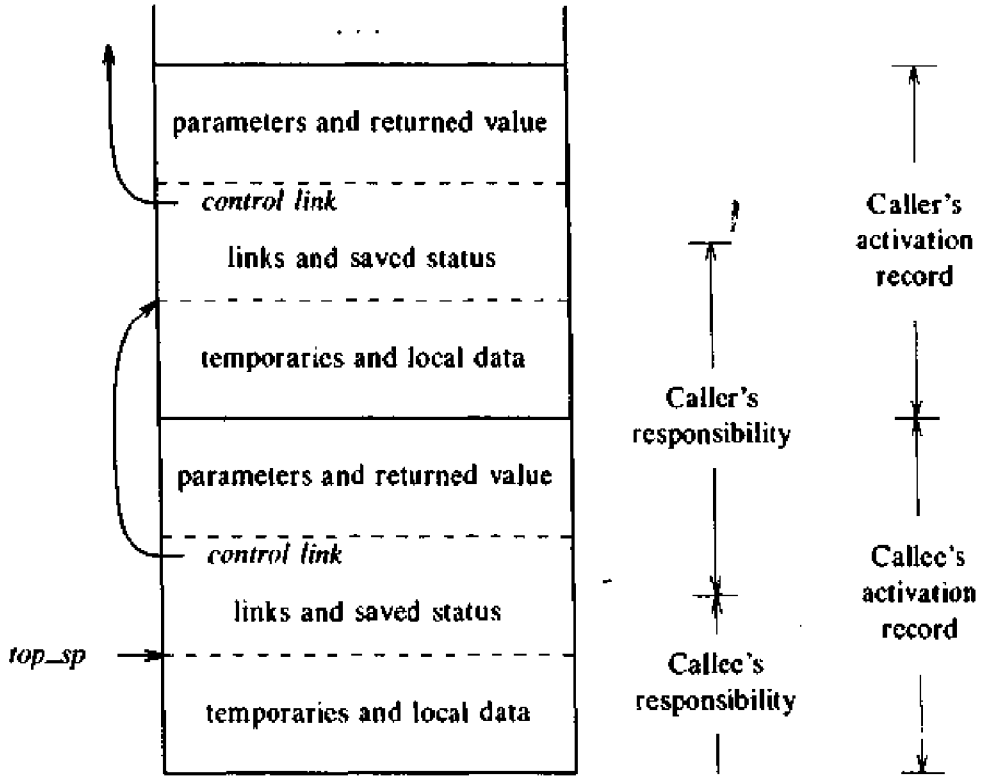


Fig. 7.14. Division of tasks between caller and callee.

The code for the callee can access its temporaries and local data using offsets from *top_sp*. The call sequence is:

1. The caller evaluates actuals.
2. The caller stores a return address and the old value of *top_sp* into the callee's activation record. The caller then increments *top_sp* to the position shown in Fig. 7.14. That is, *top_sp* is moved past the caller's local data and temporaries and the callee's parameter and status fields.
3. The callee saves register values and other status information.
4. The callee initializes its local data and begins execution.

A possible return sequence is:

1. The callee places a return value next to the activation record of the caller.
2. Using the information in the status field, the callee restores *top_sp* and other registers and branches to a return address in the caller's code.
3. Although *top_sp* has been decremented, the caller can copy the returned value into its own activation record and use it to evaluate an expression.

The above calling sequences allow the number of arguments of the called procedure to depend on the call. Note that, at compile time, the target code

of the caller knows the number of arguments it is supplying to the callee. Hence the caller knows the size of the parameter field. However, the target code of the callee must be prepared to handle other calls as well, so it waits until it is called, and then examines the parameter field. Using the organization of Fig. 7.14, information describing the parameters must be placed next to the status field so the callee can find it. For example, consider the standard library function `printf` in C. The first argument of `printf` specifies the nature of the remaining arguments, so once `printf` can locate the first argument, it can find the remaining ones.

Variable-Length Data

A common strategy for handling variable-length data is suggested in Fig. 7.15, where procedure p has three local arrays. The storage for these arrays is not part of the activation record for p; only a pointer to the beginning of each array appears in the activation record. The relative addresses of these pointers are known at compile time, so the target code can access array elements through the pointers.

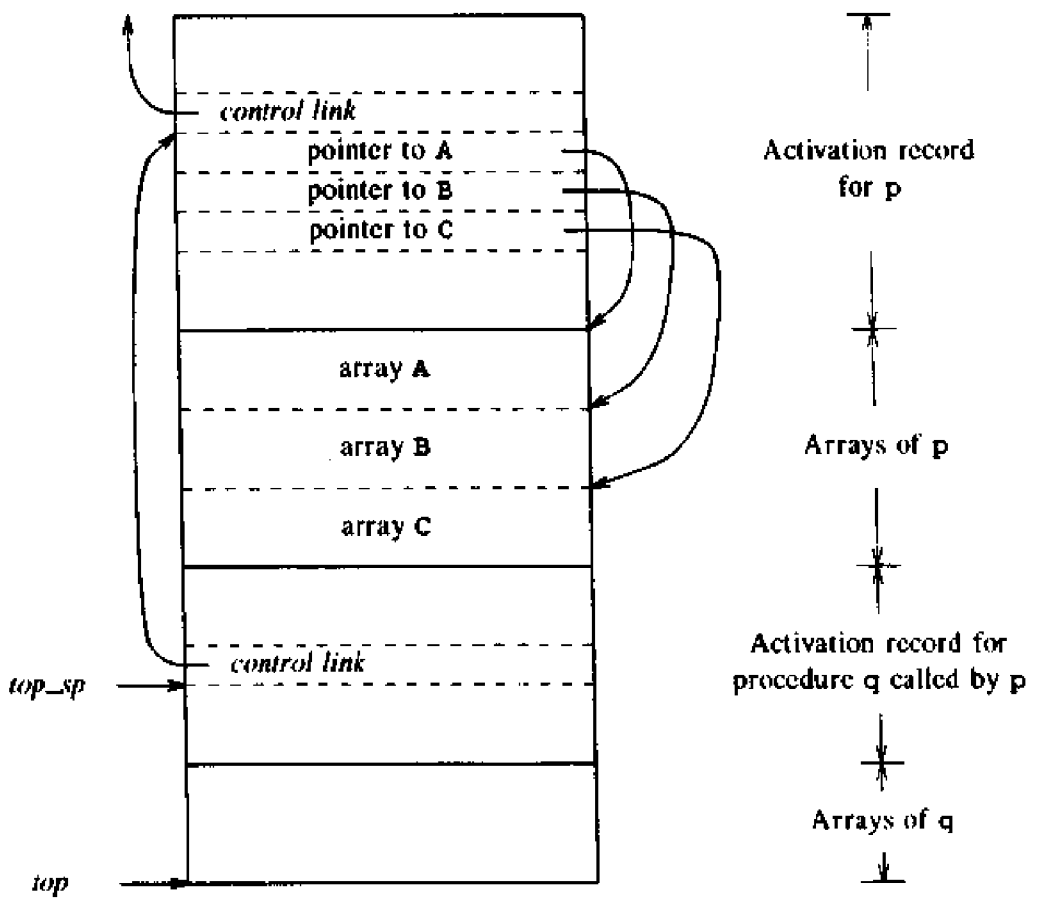


Fig. 7.15. Access to dynamically allocated arrays.

Also shown in Fig. 7.15 is a procedure q called by p. The activation record for q begins after the arrays of p, and the variable-length arrays of q begin beyond that.

Access to data on the stack is through two pointers, *top* and *top_sp*. The first of these marks the actual top of the stack; it points to the position at which the next activation record will begin. The second is used to find local data. For consistency with the organization of Fig. 7.14, suppose *top_sp* points to the end of the machine-status field. In Fig. 7.15, *top_sp* points to the end of this field in the activation record for q. Within the field is a control link to the previous value of *top_sp* when control was in the calling activation of p.

The code to reposition *top* and *top_sp* can be generated at compile time, using the sizes of the fields in the activation records. When q returns, the new value of *top* is *top_sp* minus the length of the machine-status and parameter fields in q's activation record. This length is known at compile time, at least to the caller. After adjusting *top*, the new value of *top_sp* can be copied from the control link of q.

Dangling References

Whenever storage can be deallocated, the problem of dangling references arises. A *dangling reference* occurs when there is a reference to storage that has been deallocated. It is a logical error to use dangling references, since the the value of deallocated storage is undefined according to the semantics of most languages. Worse, since that storage may later be allocated to another datum, mysterious bugs can appear in programs with dangling references.

Example 7.6. Procedure `dangle` in the C program of Fig. 7.16 returns a pointer to the storage bound to the local name `i`. The pointer is created by the operator `&` applied to `i`. When control returns to `main` from `dangle`, the storage for locals is freed and can be used for other purposes. Since `p` in `main` refers to this storage, the use of `p` is a dangling reference. □

An example in Section 7.7 involves deallocation under program control.

```
main()
{
    int *p;
    p = dangle();
}
int *dangle()
{
    int i = 23;
    return &i;
}
```

Fig. 7.16. A C program that leaves p pointing to deallocated storage.

Heap Allocation

The stack allocation strategy discussed above cannot be used if either of the following is possible.

1. The values of local names must be retained when an activation ends.
2. A called activation outlives the caller. This possibility cannot occur for those languages where activation trees correctly depict the flow of control between procedures.

In each of the above cases, the deallocation of activation records need not occur in a last-in first-out fashion, so storage cannot be organized as a stack.

Heap allocation parcels out pieces of contiguous storage, as needed for activation records or other objects. Pieces may be deallocated in any order, so over time the heap will consist of alternate areas that are free and in use.

The difference between heap and stack allocation of activation records can be seen from Fig. 7.17 and 7.13. In Fig. 7.17, the record for an activation of procedure `r` is retained when the activation ends. The record for the new activation `q(1,9)` therefore cannot follow that for `s` physically, as it did in Fig. 7.13. Now if the retained activation record for `r` is deallocated, there will be free space in the heap between the activation records for `s` and `q(1,9)`. It is left to the heap manager to make use of this space.

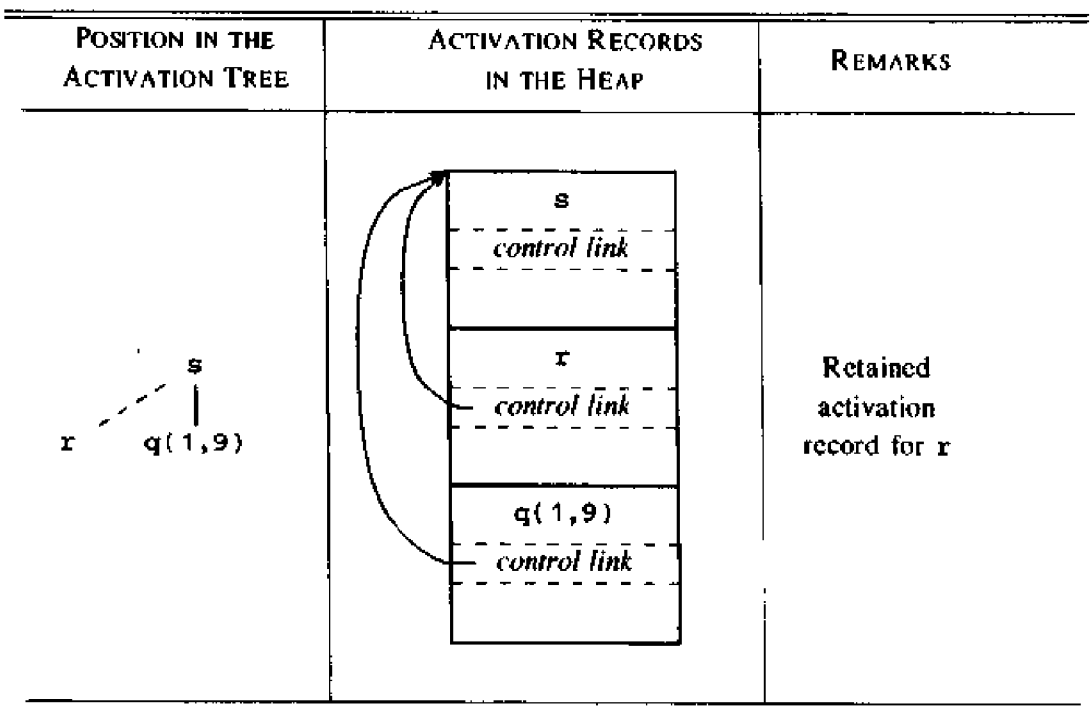


Fig. 7.17. Records for live activations need not be adjacent in a heap.

The question of efficient heap management is a somewhat specialized issue in data-structure theory; some techniques are reviewed in Section 7.8. There is generally some time and space overhead associated with using a heap manager. For efficiency reasons, it may be helpful to handle small activation records or records of a predictable size as a special case, as follows:

1. For each size of interest, keep a linked list of free blocks of that size.
2. If possible, fill a request for size s with a block of size s', where s' is the smallest size greater than or equal to s. When the block is eventually deallocated, it is returned to the linked list it came from.
3. For large blocks of storage use the heap manager.

This approach results in fast allocation and deallocation of small amounts of storage, since taking and returning a block from a linked list are efficient operations. For large amounts of storage we expect the computation to take some time to use up the storage, so the time taken by the allocator is often negligible compared with the time taken to do the computation.

7.4 ACCESS TO NONLOCAL NAMES

The storage-allocation strategies of the last section are adapted in this section to permit access to nonlocal names. Although the discussion is based on stack allocation of activation records, the same ideas apply to heap allocation.

The scope rules of a language determine the treatment of references to nonlocal names. A common rule, called the *lexical-* or *static-scope rule*, determines the declaration that applies to a name by examining the program text alone. Pascal, C, and Ada are among the many languages that use lexical scope, with an added "most closely nested" stipulation that is discussed below. An alternative rule, called the *dynamic-scope rule*, determines the declaration applicable to a name at run time, by considering the current activations. Lisp, APL, and Snobol are among the languages that use dynamic scope.

We begin with blocks and the "most closely nested" rule. Then, we consider nonlocal names in languages like C, where scopes are lexical, where all nonlocal names may be bound to statically allocated storage, and where no nested procedure declarations are allowed.

In languages like Pascal that have nested procedures and lexical scope, names belonging to different procedures may be part of the environment at a given time. We discuss two ways of finding the activation records containing the storage bound to nonlocal names: access links and displays.

A final subsection discusses the implementation of dynamic scope.

Blocks

A block is a statement containing its own local data declarations. The concept of a block originated with Algol. In C, a block has the syntax

{ *declarations statements* }

A characteristic of blocks is their nesting structure. Delimiters mark the beginning and end of a block. C uses the braces { and } as delimiters, while the Algol tradition is to use `begin` and `end`. Delimiters ensure that one block is either independent of another, or is nested inside the other. That is, it is not possible for two blocks B_1 and B_2 to overlap in such a way that first B_1 begins, then B_2, but B_1 ends before B_2. This nesting property is sometimes referred to as *block structure*.

The scope of a declaration in a block-structured language is given by the *most closely nested* rule:

1. The scope of a declaration in a block B includes B.
2. If a name x is not declared in a block B, then an occurrence of x in B is in the scope of a declaration of x in an enclosing block B' such that
   i) B' has a declaration of x, and
   ii) B' is more closely nested around B than any other block with a declaration of x.

By design, each declaration in Fig. 7.18 initializes the declared name to the number of the block that it appears in. The scope of the declaration of b in B_0 does not include B_1, because b is redeclared in B_1, indicated by $B_0 - B_1$ in the figure. Such a gap is called a *hole* in the scope of the declaration.

The most closely nested scope rule is reflected in the output of the program in Fig. 7.18. Control flows to a block from the point just before it, and flows from the block to the point just after it in the source text. The print statements are therefore executed in the order B_2, B_3, B_1, and B_0, the order in which control leaves the blocks. The values of a and b in these blocks are:

```
2 1
0 3
0 1
0 0
```

Block structure can be implemented using stack allocation. Since the scope of a declaration does not extend outside the block in which it appears, the space for the declared name can be allocated when the block is entered, and deallocated when control leaves the block. This view treats a block as a "parameterless procedure," called only from the point just before the block and returning only to the point just after the block. The nonlocal environment for a block can be maintained using the techniques for procedures later in this section. Note, however, that blocks are simpler than procedures because no parameters are passed and because the flow of control from and to a block closely follows the static program text.[5]

[5] A jump out of a block into an enclosing block can be implemented by popping activation records for the intervening blocks. A jump into a block is permitted by some languages. Before control is transferred in this way, activation records have to be set up for the intervening blocks. The language semantics determines how local data in these activation records is initialized.

```
main()
{
    int a = 0;
    int b = 0;
    {
        int b = 1;
        {
            int a = 2;
            printf("%d %d\n", a, b);
        }
        {
            int b = 3;
            printf("%d %d\n", a, b);
        }
        printf("%d %d\n", a, b);
    }
    printf("%d %d\n", a, b);
}
```

B_0 B_1 B_2 B_3

DECLARATION	SCOPE
int a = 0;	$B_0 - B_2$
int b = 0;	$B_0 - B_3$
int b = 1;	$B_1 - B_3$
int a = 2;	B_2
int b = 3;	B_3

Fig. 7.18. Blocks in a C program.

An alternative implementation is to allocate storage for a complete procedure body at one time. If there are blocks within the procedure, then allowance is made for the storage needed for declarations within the blocks. For block B_0 in Fig. 7.18, we can allocate storage as in Fig. 7.19. Subscripts on locals a and b identify the blocks that the locals are declared in. Note that a_2 and b_3 may be assigned the same storage because they are in blocks that are not alive at the same time.

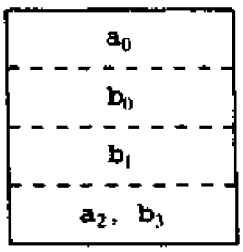


Fig. 7.19. Storage for names declared in Fig. 7.18.

In the absence of variable-length data, the maximum amount of storage needed during any execution of a block can be determined at compile time. (Variable-length data can be handled using pointers as in Section 7.3.) In making this determination, we conservatively assume that all control paths in

the program can indeed be taken. That is, we assume that both the then- and else-parts of a conditional statement can be executed, and that all statements within a while loop can be reached.

Lexical Scope Without Nested Procedures

The lexical-scope rules for C are simpler than those of Pascal, discussed next, because procedure definitions cannot be nested in C. That is, a procedure definition cannot appear within another. As in Fig. 7.20, a C program consists of a sequence of declarations of variables and procedures (C calls them functions). If there is a nonlocal reference to a name `a` in some function, then `a` must be declared outside any function. The scope of a declaration outside a function consists of the function bodies that follow the declaration, with holes if the name is redeclared within a function. In Fig. 7.20, nonlocal occurrences of `a` in `readarray`, `partition`, and `main` refer to the array declared on line 1:

```
(1) int a[11];
(2) readarray() { ... a ... }
(3) int partition(y,z) int y, z; { ... a ... }
(4) quicksort(m,n) int m, n; { ... }
(5) main() { ... a ... }
```

Fig. 7.20. C program with nonlocal occurrences of `a`.

In the absence of nested procedures, the stack-allocation strategy for local names in Section 7.3 can be used directly for a lexically scoped language like C. Storage for all names declared outside any procedures can be allocated statically. The position of this storage is known at compile time, so if a name is nonlocal in some procedure body, we simply use the statically determined address. Any other name must be a local of the activation at the top of the stack, accessible through the *top* pointer. Nested procedures cause this scheme to fail because a nonlocal may then refer to data deep in the stack, as discussed below.

An important benefit of static allocation for nonlocals is that declared procedures can freely be passed as parameters and returned as results (a function is passed in C by passing a pointer to it). With lexical scope and without nested procedures, any name nonlocal to one procedure is nonlocal to all procedures. Its static address can be used by all procedures, regardless of how they are activated. Similarly, if procedures are returned as results, nonlocals in the returned procedure refer to the storage statically allocated for them.

For example, consider the Pascal program in Fig. 7.21. All occurrences of name `m`, shown circled in Fig. 7.21, are in the scope of the declaration on line 2. Since `m` is nonlocal to all procedures in the program, its storage can be allocated statically. Whenever procedures `f` and `g` are executed, they can use

```
(1)  program pass(input, output);
(2)     var (m): integer;

(3)     function f(n : integer) : integer;
(4)        begin f := (m)* n end { f };

(5)     function g(n : integer) : integer;
(6)        begin g := (m)* n end { g };

(7)     procedure b(function h(n : integer) : integer);
(8)        begin write(h(2)) end { b };

(9)     begin
(10)       (m):= 0;
(11)       b(f); b(g); writeln
(12)    end.
```

Fig. 7.21. Pascal program with nonlocal occurrences of m.

the static address to access the value of m. The fact that f and g are passed as parameters only affects when they are activated; it does not affect how they access the value of m.

In more detail, the call b(f) on line 11 associates the function f with the formal parameter h of the procedure b. So when the formal h is called on line 8, in write(h(2)), the function f is activated. The activation of f returns 2 because nonlocal m has value 0 and formal n has value 2. Next in the execution, the call b(g) associates g with h; this time, a call of h activates g. The output of the program is

```
2           0
```

Lexical Scope with Nested Procedures

A nonlocal occurrence of a name a in a Pascal procedure is in the scope of the most closely nested declaration of a in the static program text.

The nesting of procedure definitions in the Pascal program of Fig. 7.22 is indicated by the following indentation:

```
sort
    readarray
    exchange
    quicksort
        partition
```

The occurrence of a on line 15 in Fig. 7.22 is within function partition, which is nested within procedure quicksort. The most closely nested declaration of a is on line 2 in the procedure consisting of the entire program. The most closely nested rule applies to procedure names as well. The

```
(1)  program sort(input, output);
(2)      var a : array [0..10] of integer;
(3)          x : integer;
(4)      procedure readarray;
(5)          var i : integer;
(6)          begin ... a ... end { readarray };
(7)      procedure exchange( i, j: integer);
(8)          begin
(9)              x := a[i]; a[i] := a[j]; a[j] := x
(10)         end { exchange } ;
(11)     procedure quicksort(m, n: integer);
(12)         var k, v : integer;
(13)         function partition(y, z: integer): integer;
(14)             var   i, j : integer;
(15)             begin ... a ...
(16)                   ... v ...
(17)                   ... exchange(i,j); ...
(18)             end { partition } ;
(19)         begin ... end { quicksort };
(20)     begin ... end { sort } .
```

Fig. 7.22. A Pascal program with nested procedures.

procedure `exchange`, called by `partition` on line 17, is nonlocal to `partition`. Applying the rule, we first check if `exchange` is defined within `quicksort`; since it is not, we look for it in the main program `sort`.

Nesting Depth

The notion of *nesting depth* of a procedure is used below to implement lexical scope. Let the name of the main program be at nesting depth 1; we add 1 to the nesting depth as we go from an enclosing to an enclosed procedure. In Fig. 7.22, procedure `quicksort` on line 11 is at nesting depth 2, while `partition` on line 13 is at nesting depth 3. With each occurrence of a name, we associate the nesting depth of the procedure in which it is declared. The occurrences of `a`, `v`, and `i` on lines 15-17 in `partition` therefore have nesting depths 1, 2, and 3, respectively.

Access Links

A direct implementation of lexical scope for nested procedures is obtained by adding a pointer called an *access link* to each activation record. If procedure

p is nested immediately within q in the source text, then the access link in an activation record for p points to the access link in the record for the most recent activation of q.

Snapshots of the run-time stack during an execution of the program in Fig. 7.22 are shown in Fig. 7.23. Again, to save space in the figure, only the first letter of each procedure name is shown. The access link for the activation of `sort` is empty, because there is no enclosing procedure. The access link for each activation of `quicksort` points to the record for `sort`. Note in Fig. 7.23(c) that the access link in the activation record for `partition(1,3)` points to the access link in the record of the most recent activation of `quicksort`, namely `quicksort(1,3)`.

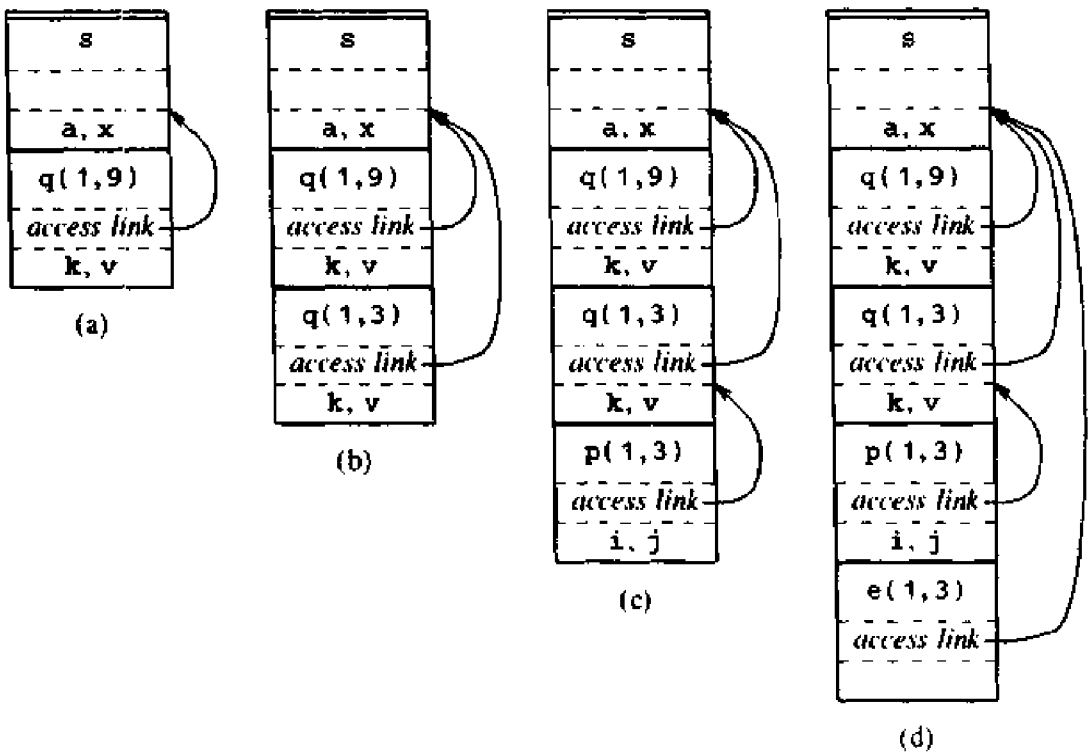


Fig. 7.23. Access links for finding storage for nonlocals.

Suppose procedure p at nesting depth n_p refers to a nonlocal a with nesting depth $n_a \leq n_p$. The storage for a can be found as follows.

1. When control is in p, an activation record for p is at the top of the stack. Follow $n_p - n_a$ access links from the record at the top of the stack. The value of $n_p - n_a$ can be precomputed at compile time. If the access link in one record points to the access link in another, then a link can be followed by performing a single indirection operation.

2. After following $n_p - n_a$ links, we reach an activation record for the procedure that a is local to. As discussed in the last section, its storage is at

a fixed offset relative to a position in the record. In particular, the offset can be relative to the access link.

Hence, the address of nonlocal a in procedure p is given by the following pair computed at compile time and stored in the symbol table:

($n_p - n_a$, offset within activation record containing a)

The first component gives the number of access links to be traversed.

For example, on lines 15-16 in Fig. 7.22, the procedure `partition` at nesting depth 3 references nonlocals `a` and `v` at nesting depths 1 and 2, respectively. The activation records containing the storage for these nonlocals are found by following $3-1=2$ and $3-2=1$ access links, respectively, from the record for `partition`.

The code to set up access links is part of the calling sequence. Suppose procedure p at nesting depth n_p calls procedure x at nesting depth n_x. The code for setting up the access link in the called procedure depends on whether or not the called procedure is nested within the caller.

1. Case $n_p < n_x$. Since the called procedure x is nested more deeply than p, it must be declared within p, or it would not be accessible to p. This case occurs when `sort` calls `quicksort` in Fig. 7.23(a) and when `quicksort` calls `partition` in Fig. 7.23(c). In this case, the access link in the called procedure must point to the access link in the activation record of the caller just below in the stack.

2. Case $n_p \geq n_x$. From the scope rules, the enclosing procedures at nesting depths 1, 2, . . . , $n_x - 1$ of the called and calling procedures must be the same, as when `quicksort` calls itself in Fig. 7.23(b) and when `partition` calls `exchange` in Fig. 7.23(d). Following $n_p - n_x + 1$ access links from the caller we reach the most recent activation record of the procedure that statically encloses both the called and calling procedures most closely. The access link reached is the one to which the access link in the called procedure must point. Again, $n_p - n_x + 1$ can be computed at compile time.

Procedure Parameters

Lexical scope rules apply even when a nested procedure is passed as a parameter. The function `f` on lines 6-7 of the Pascal program in Fig. 7.24 has a nonlocal `m`; all occurrences of `m` are shown circled. On line 8, procedure `c` assigns `0` to `m` and then passes `f` as a parameter to `b`. Note that the scope of the declaration of `m` on line 5 does not include the body of `b` on lines 2-3.

Within the body of `b`, the statement `writeln(h(2))` activates `f` because the formal `h` refers to `f`. That is, `writeln` prints the result of the call `f(2)`.

How do we set up the access link for the activation of `f`? The answer is that a nested procedure that is passed as a parameter must take its access link along with it, as shown in Fig. 7.25. When procedure `c` passes `f`, it

```
(1)  program param(input, output);
(2)     procedure b(function h(n:integer): integer);
(3)        begin writeln(h(2)) end { b };
(4)     procedure c;
(5)        var (m): integer;
(6)        function f(n : integer) : integer;
(7)           begin f := (m)+ n end { f };
(8)        begin (m):= 0; b(f) end { c };
(9)     begin
(10)     c
(11)    end.
```

Fig. 7.24. An access link must be passed with actual parameter `f`.

determines an access link for `f`, just as it would if it were calling `f`. This link is passed along with `f` to `b`. Subsequently, when `f` is activated from within `b`, the link is used to set up the access link in the activation record for `f`.

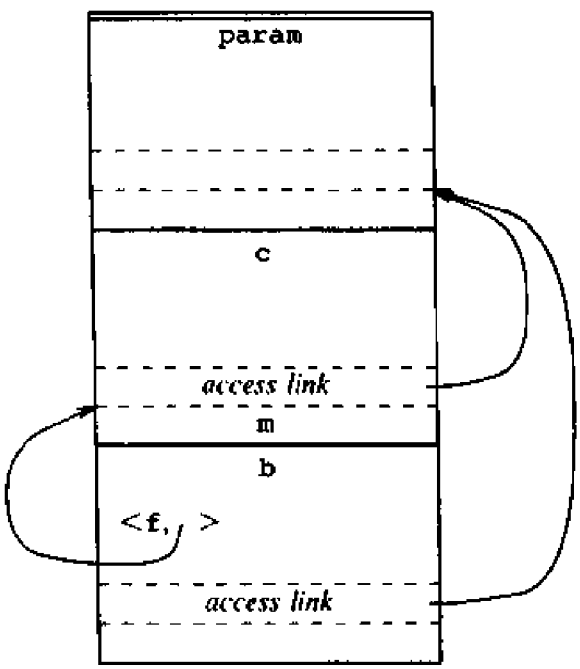


Fig. 7.25. Actual procedure parameter `f` carries its access link along.

Displays

Faster access to nonlocals than with access links can be obtained using an array d of pointers to activation records, called a *display*. We maintain the display so that storage for a nonlocal a at nesting depth i is in the activation record pointed to by display element $d[i]$.

Suppose control is in an activation of a procedure p at nesting depth j. Then, the first $j-1$ elements of the display point to the most recent activations of the procedures that lexically enclose procedure p, and $d[j]$ points to the activation of p. Using a display is generally faster than following access links because the activation record holding a nonlocal is found by accessing an element of d and then following just one pointer.

A simple arrangement for maintaining the display uses access links in addition to the display. As part of the call and return sequences, the display is updated by following the chain of access links. When the link to an activation record at nesting depth n is followed, display element $d[n]$ is set to point to that activation record. In effect, the display duplicates the information in the chain of access links.

The above simple arrangement can be improved upon. The method illustrated in Fig. 7.26 requires less work at procedure entry and exit in the usual case when procedures are not passed as parameters. In Fig. 7.26 the display consists of a global array, separate from the stack. The snapshots in the figure refer to an execution of the source text in Fig. 7.22. Again, only the first letter of each procedure name is shown.

Figure 7.26(a) shows the situation just before activation `q(1,3)` begins. Since `quicksort` is at nesting depth 2, display element $d[2]$ is affected when a new activation of `quicksort` begins. The effect of the activation `q(1,3)` on $d[2]$ is shown in Fig 7.26(b), where $d[2]$ now points to the new activation record; the old value of $d[2]$ is saved within the new activation record.[6] The saved value will be needed later to restore the display to its state in Fig. 7.26(a) when control returns to the activation `q(1,9)`.

The display changes when a new activation occurs, and it must be reset when control returns from the new activation. The scope rules of Pascal and other lexically scoped languages allow the display to be maintained by the following steps. We discuss only the easier case in which procedures are not passed as parameters (see Exercise 7.8). When a new activation record for a procedure at nesting depth i is set up, we

1. save the value of $d[i]$ in the new activation record and
2. set $d[i]$ to point to the new activation record.

Just before an activation ends, $d[i]$ is reset to the saved value.

[6] Note that `q(1,9)` also saved $d[2]$, although it happens that the second display element had never before been used and does not need to be restored. It is easier for all calls of q to store $d[2]$ than to decide at run time whether such a store is necessary.

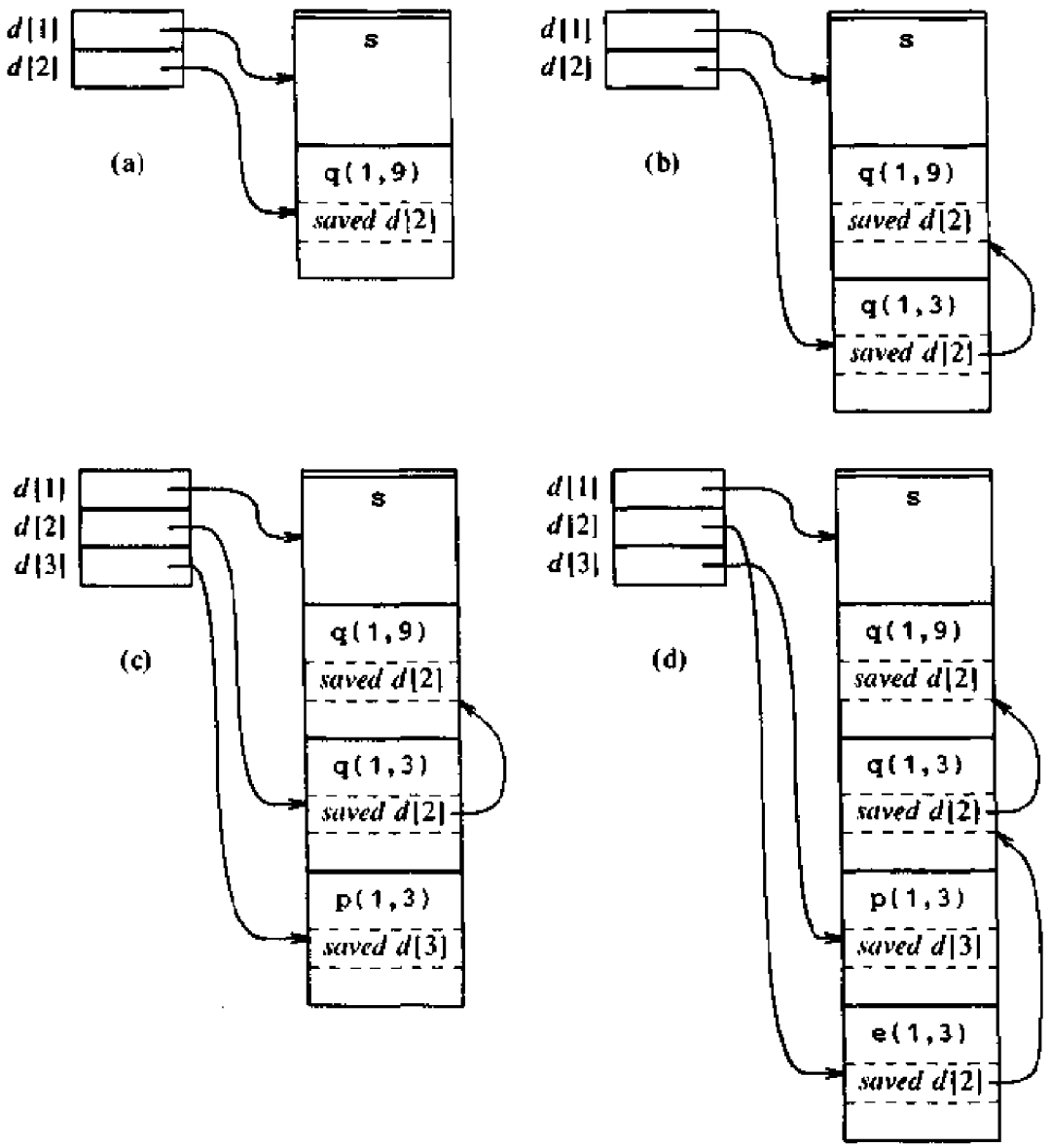


Fig. 7.26. Maintaining the display when procedures are not passed as parameters.

These steps are justified as follows. Suppose a procedure at nesting depth j calls a procedure at depth i. There are two cases, depending on whether or not the called procedure is nested within the caller in the source text, as in the discussion of access links.

1. Case $j < i$. Then $i = j + 1$ and the called procedure is nested within the caller. The first j elements of the display therefore do not need to be changed, and we set $d[i]$ to the new activation record. This case is illustrated in Fig. 7.26(a) when `sort` calls `quicksort` and also when `quicksort` calls `partition` in Fig. 7.26(c).

2. Case $j \geq i$. Again, the enclosing procedures at nesting depths $1, 2, \ldots, i-1$ of the called and calling procedure must be the same. Here, we save the old value of $d[i]$ in the new activation record, and make $d[i]$ point to the new activation record. The display is maintained correctly because the first $i-1$ elements are left as is.

An example of Case 2, with $i = j = 2$, occurs when `quicksort` is called recursively in Fig. 7.26(b). A more interesting example occurs when activation `p(1,3)` at nesting depth 3 calls `e(1,3)` at depth 2, and their enclosing procedure is `s` at depth 1, as in Fig. 7.26(d). (The program is in Fig. 7.22.) Note that when `e(1,3)` is called, the value of $d[3]$ belonging to `p(1,3)` is still in the display, although it cannot be accessed while control is in `e`. Should `e` call another procedure at depth 3, that procedure will store $d[3]$ and restore it on returning to `e`. We can thus show that each procedure sees the correct display for all depths up to its own depth.

There are several places where a display can be maintained. If there are enough registers, then the display, pictured as an array, can be a collection of registers. Note that the compiler can determine the maximum length of this array; it is the maximum nesting depth of procedures in the program. Otherwise, the display can be kept in statically allocated memory and all references to activation records begin by using indirect addressing through the appropriate display pointer. This approach is reasonable on a machine with indirect addressing, although each indirection costs a memory cycle. Another possibility is to store the display on the run-time stack itself, and to create a new copy at each procedure entry.

Dynamic Scope

Under dynamic scope, a new activation inherits the existing bindings of nonlocal names to storage. A nonlocal name `a` in the called activation refers to the same storage that it did in the calling activation. New bindings are set up for local names of the called procedure; the names refer to storage in the new activation record.

The program in Fig. 7.27 illustrates dynamic scope. Procedure `show` on lines 3-4 writes the value of nonlocal `x`. Under lexical scope in Pascal, the nonlocal `x` is in the scope of the declaration on line 2, so the output of the program is

```
0.250 0.250
0.250 0.250
```

However, under dynamic scope, the output is

```
0.250 0.125
0.250 0.125
```

When `show` is called on lines 10-11 in the main program, `0.250` is written

```
(1)  program dynamic(input,output);
(2)      var r : real;
(3)      procedure show;
(4)          begin write( r : 5:3 ) end;
(5)      procedure small;
(6)          var r : real;
(7)          begin r := 0.125; show end;
(8)      begin
(9)          r := 0.25;
(10)         show; small; writeln;
(11)         show; small; writeln
(12)     end.
```

Fig. 7.27. The output depends on whether lexical or dynamic scope is used.

because the variable `r` local to the main program is used. However, when `show` is called on line 7 from within `small`, `0.125` is written because the variable `r` local to `small` is used.

The following two approaches to implementing dynamic scope bear some resemblance to the use of access links and displays, respectively, in the implementation of lexical scope.

1. *Deep access.* Conceptually, dynamic scope results if access links point to the same activation records that control links do. A simple implementation is to dispense with access links and use the control link to search into the stack, looking for the first activation record containing storage for the nonlocal name. The term *deep access* comes from the fact that the search may go "deep" into the stack. The depth to which the search may go depends on the input to the program and cannot be determined at compile time.

2. *Shallow access.* Here the idea is to hold the current value of each name in statically allocated storage. When a new activation of a procedure `p` occurs, a local name `n` in `p` takes over the storage statically allocated for `n`. The previous value of `n` can be saved in the activation record for `p` and must be restored when the activation of `p` ends.

The tradeoff between the two approaches is that deep access takes longer to access a nonlocal, but there is no overhead associated with beginning and ending an activation. Shallow access on the other hand allows nonlocals to be looked up directly, but time is taken to maintain these values when activations begin and end. When functions are passed as parameters and returned as results, a more straightforward implementation is obtained with deep access.

7.5 PARAMETER PASSING

When one procedure calls another, the usual method of communication between them is through nonlocal names and through parameters of the called procedure. Both nonlocals and parameters are used by the procedure in Fig. 7.28 to exchange the values of `a[i]` and `a[j]`. Here the array `a` is nonlocal to the procedure `exchange`, and `i` and `j` are parameters.

```
(1)  procedure exchange(i, j: integer);
(2)      var x : integer;
(3)      begin
(4)          x := a[i]; a[i] := a[j]; a[j] := x
(5)      end
```

Fig. 7.28. The Pascal procedure swap with nonlocals and parameters.

Several common methods for associating actual and formal parameters are discussed in this section. They are: call-by-value, call-by-reference, copy-restore, call-by-name, and macro expansion. It is important to know the parameter passing method a language (or compiler) uses, because the result of a program can depend on the method used.

Why so many methods? The different methods arise from differing interpretations of what an expression represents. In an assignment like

```
a[i] := a[j]
```

expression `a[j]` represents a value, while `a[i]` represents a storage location into which the value of `a[j]` is placed. The decision of whether to use the location or the value represented by an expression is determined by whether the expression appears on the left or right, respectively, of the assignment symbol. As in Chapter 2, the term *l*-value refers to the storage represented by an expression, and *r*-value to the value contained in the storage. The prefixes *l*- and *r*- come from "left" and "right" side of an assignment.

Differences between parameter-passing methods are based primarily on whether an actual parameter represents an *r*-value, an *l*-value, or the text of the actual parameter itself.

Call-by-Value

This is, in a sense, the simplest possible method of passing parameters. The actual parameters are evaluated and their *r*-values are passed to the called procedure. Call-by-value is used in C, and Pascal parameters are usually passed this way. All the programs so far in this chapter have relied on this method for passing parameters. Call-by-value can be implemented as follows.

1. A formal parameter is treated just like a local name, so the storage for the formals is in the activation record of the called procedure.

2. The caller evaluates the actual parameters and places their *r*-values in the storage for the formals.

A distinguishing feature of call-by-value is that operations on the formal parameters do not affect values in the activation record of the caller. If the keyword `var` on line 3 in Fig. 7.29 is left out, Pascal will pass `x` and `y` by value to the procedure `swap`. The call `swap(a,b)` on line 12 then leaves the values of `a` and `b` undisturbed. Under call-by-value, the effect of the call `swap(a,b)` is equivalent to the sequence of steps

```
   x := a
   y := b
temp := x
   x := y
   y := temp
```

where `x`, `y`, and `temp` are local to `swap`. Although these assignments change the values of the locals `x`, `y`, and `temp`, the changes are lost when control returns from the call and the activation record for `swap` is deallocated. The call therefore has no effect on the activation record of the caller.

```
(1) program reference(input,output);
(2) var a, b: integer;
(3) procedure swap(var x, y: integer);
(4)     var temp : integer;
(5)     begin
(6)         temp := x;
(7)            x := y;
(8)            y := temp
(9)     end;
(10) begin
(11)     a := 1; b := 2;
(12)     swap(a,b);
(13)     writeln('a =', a); writeln('b =', b)
(14) end.
```

Fig. 7.29. Pascal program with procedure `swap`.

A procedure called by value can affect its caller either through nonlocal names (see `exchange` in Fig. 7.28) or through pointers that are explicitly passed as values. In the C program in Fig. 7.30, `x` and `y` are declared on line 2 to be pointers to integers; the `&` operator in the call `swap(&a, &b)` on line 8 results in pointers to `a` and `b` being passed to `swap`. The output of this program is

```
a is now 2, b is now 1
```

```
(1) swap(x,y)
(2) int *x, *y;
(3) {   int temp;
(4)     temp = *x; *x = *y; *y = temp;
(5) }

(6) main()
(7) {   int a = 1, b = 2;
(8)     swap( &a, &b );
(9)     printf("a is now %d, b is now %d\n",a,b);
(10) }
```

Fig. 7.30. C program using pointers in a procedure called by value.

The use of pointers in this example suggests how a compiler using call-by-reference would exchange values.

Call-by-Reference

When parameters are passed by *reference* (also known as *call-by-address* or *call-by-location*), the caller passes to the called procedure a pointer to the storage address of each actual parameter.

1. If an actual parameter is a name or an expression having an *l*-value, then that *l*-value itself is passed.
2. However, if the actual parameter is an expression, like `a+b` or `2`, that has no *l*-value, then the expression is evaluated in a new location, and the address of that location is passed.

A reference to a formal parameter in the called procedure becomes, in the target code, an indirect reference through the pointer passed to the called procedure.

Example 7.7. Consider the procedure `swap` in Fig. 7.29. A call to `swap` with actual parameters `i` and `a[i]`, that is, `swap(i, a[i])`, would have the same effect as the following sequence of steps.

1. Copy the address (*l*-values) of `i` and `a[i]` into the activation record of the called procedure, say into locations `arg1` and `arg2` corresponding to `x` and `y`, respectively.
2. Set `temp` to the contents of the location pointed to by `arg1` (i.e., set `temp` equal to I_0, where I_0 is the initial value of `i`). This step corresponds to `temp := x` on line 6 in the definition of `swap`.
3. Set the contents of the location pointed to by `arg1` to the value of the location pointed to by `arg2`; that is, `i := a[`I_0`]`. This step corresponds to `x := y` on line 7 in `swap`.

4. Set the contents of the location pointed by `arg2` equal to the value of `temp`; that is, set `a[`I_0`] := i`. This step corresponds to `y:=temp`. □

Call-by-reference is used by a number of languages; `var` parameters in Pascal are passed in this way. Arrays are usually passed by reference.

Copy-Restore

A hybrid between call-by-value and call-by-reference is *copy-restore linkage*, (also known as *copy-in copy-out*, or *value-result*).

1. Before control flows to the called procedure, the actual parameters are evaluated. The *r*-values of the actuals are passed to the called procedure as in call-by-value. In addition, however, the *l*-values of those actual parameters having *l*-values are determined before the call.

2. When control returns, the current *r*-values of the formal parameters are copied back into the *l*-values of the actuals, using the *l*-values computed before the call. Only actuals having *l*-values are copied, of course.

The first step "copies in" the values of the actuals into the activation record of the called procedure (into the storage for the formals). The second step "copies out" the final values of the formals into the activation record of the caller (into *l*-values computed from the actuals before the call).

Note that `swap(i,a[i])` works correctly using copy-restore, since the location of `a[i]` is computed and preserved by the calling program before the call. Thus, the final value of formal parameter `y`, which will be the initial value of `i`, is copied into the correct location, even though the location of `a[i]` is changed by the call (because the value of `i` changes).

Copy-restore is used by some Fortran implementations. However, others use call-by-reference. Differences between the two can show up if the called procedure has more than one way of accessing a location in the activation record of the caller. The activation set up by the call `unsafe(a)` on line 6 of Fig. 7.31 can access `a` as a nonlocal and through formal `x`. Under call-by-reference, the assignments to both `x` and `a` immediately affect `a`, so the final value of `a` is 0. Under copy-restore, however, the value 1 of actual `a` is copied into formal `x`. The final value 2 of `x` is copied out into the *l*-value of `a` just before control returns, so the final value of `a` is 2.

```
(1) program copyout(input,output);
(2)     var a : integer;
(3)     procedure unsafe(var x : integer);
(4)        begin x := 2; a := 0 end;
(5)     begin
(6)        a := 1; unsafe(a); writeln(a)
(7)     end.
```

Fig. 7.31. The output changes if call-by-reference is changed to copy-restore.

Call-by-Name

Call-by-name is traditionally defined by the *copy-rule* of Algol, which is:

1. The procedure is treated as if it were a macro; that is, its body is substituted for the call in the caller, with the actual parameters literally substituted for the formals. Such a literal substitution is called *macro-expansion* or *in-line expansion*.
2. The local names of the called procedure are kept distinct from the names of the calling procedure. We can think of each local of the called procedure being systematically renamed into a distinct new name before the macro-expansion is done.
3. The actual parameters are surrounded by parentheses if necessary to preserve their integrity.

Example 7.8. The call `swap(i, a[i])` from Example 7.7 would be implemented as though it were

```
temp := i
   i := a[i]
a[i] := temp
```

Thus, under call-by-name, `swap` sets `i` to `a[i]`, as expected, but has the unexpected result of setting `a[a[`I_0`]]` — rather than `a[`I_0`]` — to I_0, where I_0 is the initial value of `i`. This phenomenon occurs because the location of `x` in the assignment `x := temp` of `swap` is not evaluated until needed, by which time the value of `i` has already changed. A correctly working version of `swap` apparently cannot be written if call-by-name is used (see Fleck [1976]). □

Although call-by-name is primarily of theoretical interest, the conceptually related technique of in-line expansion has been suggested for reducing the running time of a program. There is a certain cost associated with setting up an activation of a procedure — space is allocated for the activation record, the machine status is saved, links are set up, and then control is transferred. When a procedure body is small, the code devoted to the calling sequences may outweigh the code in the procedure body. It may therefore be more efficient to use in-line expansion of the body into the code for the caller, even if the size of the program grows a little. In the next example, in-line expansion is applied to a procedure called by value.

Example 7.9. Suppose that the function `f` in the assignment

```
x := f(A) + f(B)
```

is called by value. Here the actual parameters A and B are expressions. Substituting expressions A and B for each occurrence of the formal parameter in the body of `f` leads to call-by-name; recall `a[i]` in the last example.

Fresh temporary variables can be used to force the evaluation of the actual parameters before execution of the procedure body:

```
t1 := A;
t2 := B;
t3 := f(t1);
t4 := f(t2);
 x := t3 + t4;
```

Now in-line expansion will replace all occurrences of the formal by t_1 and t_2 when the first and second calls, respectively, are expanded.[7] □

The usual implementation of call-by-name is to pass to the called procedure parameterless subroutines, commonly called *thunks*, that can evaluate the *l*-value or *r*-value of the actual parameter. Like any procedure passed as a parameter in a language using lexical scope, a thunk carries an access link with it, pointing to the current activation record for the calling procedure.

7.6 SYMBOL TABLES

A compiler uses a symbol table to keep track of scope and binding information about names. The symbol table is searched every time a name is encountered in the source text. Changes to the table occur if a new name or new information about an existing name is discovered.

A symbol-table mechanism must allow us to add new entries and find existing entries efficiently. The two symbol-table mechanisms presented in this section are linear lists and hash tables. We evaluate each scheme on the basis of the time required to add n entries and make e inquiries. A linear list is the simplest to implement, but its performance is poor when e and n get large. Hashing schemes provide better performance for somewhat greater programming effort and space overhead. Both mechanisms can be adapted readily to handle the most closely nested scope rule.

It is useful for a compiler to be able to grow the symbol table dynamically, if necessary, at compile time. If the size of the symbol table is fixed when the compiler is written, then the size must be chosen large enough to handle any source program that might be presented. Such a fixed size is likely to be too large for most, and inadequate for some, programs.

Symbol-Table Entries

Each entry in the symbol table is for the declaration of a name. The format of entries does not have to be uniform, because the information saved about a name depends on the usage of the name. Each entry can be implemented as a

[7] There are hidden costs associated with temporary variables. They may cause extra space to be allocated in an activation record. If locals in the activation record are initialized, then extra temporaries result in time being wasted as well.

record consisting of a sequence of consecutive words of memory. To keep symbol-table records uniform, it may be convenient for some of the information about a name to be kept outside the table entry, with only a pointer to this information stored in the record.

Information is entered into the symbol table at various times. Keywords are entered into the table initially, if at all. The lexical analyzer in Section 3.4 looks up sequences of letters and digits in the symbol table to determine if a reserved keyword or a name has been collected. With this approach, keywords must be in the symbol table before lexical analysis begins. Alternatively, if the lexical analyzer intercepts reserved keywords, then they need not appear in the symbol table. If the language does not reserve keywords, then it is essential that keywords be entered into the symbol table with a warning of their possible use as a keyword.

The symbol-table entry itself can be set up when the role of a name becomes clear, with the attribute values being filled in as the information becomes available. In some cases, the entry can be initiated from the lexical analyzer as soon as a name is seen in the input. More often, one name may denote several different objects, perhaps even in the same block or procedure. For example, the C declarations

```
int    x;
struct x { float y, z; };
```
(7.1)

use x as both an integer and as the tag of a structure with two fields. In such cases, the lexical analyzer can only return to the parser the name itself (or a pointer to the lexeme forming that name), rather than a pointer to the symbol-table entry. The record in the symbol table is created when the syntactic role played by this name is discovered. For the declarations in (7.1), two symbol-table entries for x would be created; one with x as an integer and one as a structure.

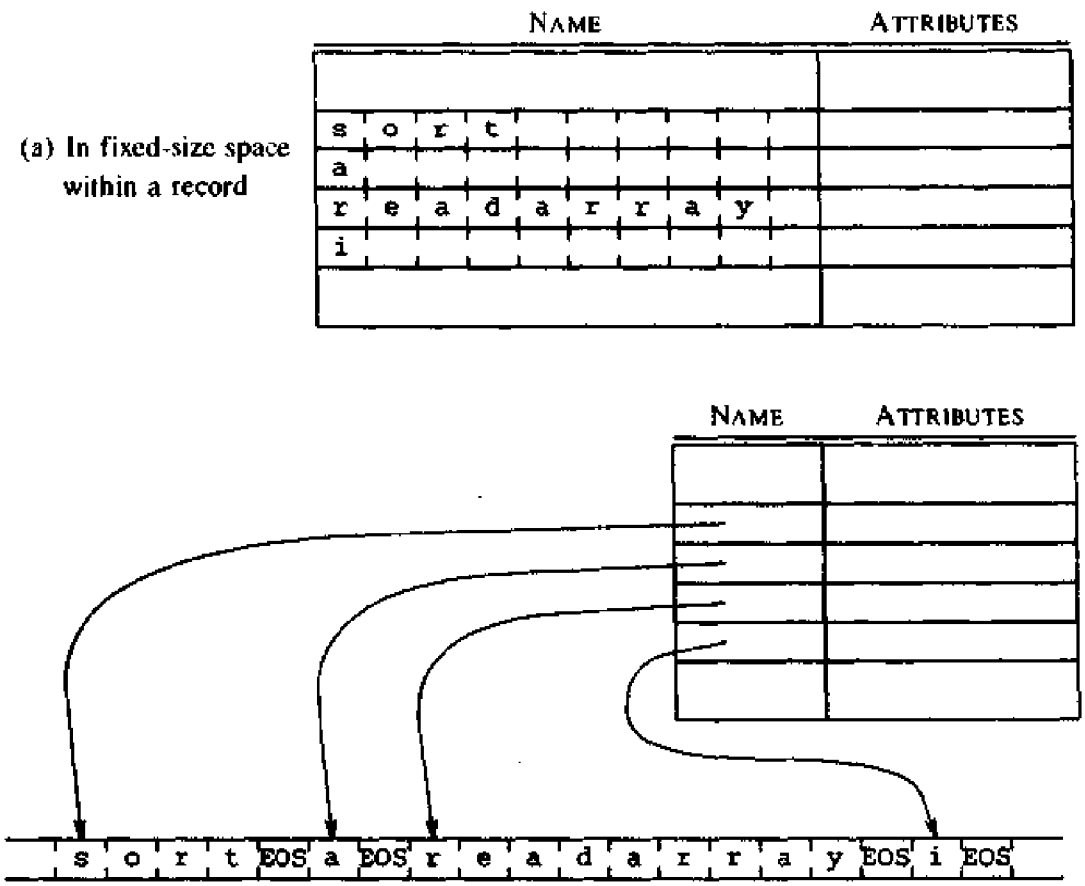
Attributes of a name are entered in response to declarations, which may be implicit. Labels are often identifiers followed by a colon, so one action associated with recognizing such an identifier may be to enter this fact into the symbol table. Similarly, the syntax of procedure declarations specifies that certain identifiers are formal parameters.

Characters in a Name

As in Chapter 3, there is a distinction between the token **id** for an identifier or name, the lexeme consisting of the character string forming the name, and the attributes of the name. Strings of characters may be unwieldy to work with, so compilers often use some fixed-length representation of the name rather than the lexeme. The lexeme is needed when a symbol-table entry is set up for the first time, and when we look up a lexeme found in the input to determine whether it is a name that has already appeared. A common representation of a name is a pointer to a symbol-table entry for it.

If there is a modest upper bound on the length of a name, then the characters in the name can be stored in the symbol-table entry, as in Fig. 7.32(a). If there is no limit on the length of a name, or if the limit is rarely reached, the indirect scheme of Fig. 7.32(b) can be used. Rather than allocating in each symbol-table entry the maximum possible amount of space to hold a lexeme, we can utilize space more efficiently if there is only space for a pointer in a symbol-table entry. In the record for a name, we place a pointer to a separate array of characters (the *string table*) giving the position of the first character of the lexeme. The indirect scheme of Fig. 7.32(b) permits the size of the name field of the symbol-table entry itself to remain a constant.

The complete lexeme constituting a name must be stored to ensure that all uses of the same name can be associated with the same symbol-table record. We must, however, distinguish among occurrences of the same lexeme that are in the scopes of different declarations.



(b) In a separate array

Fig. 7.32. Storing the characters of a name.

Storage Allocation Information

Information about the storage locations that will be bound to names at run time is kept in the symbol table. Consider names with static storage first. If the target code is assembly language, we can let the assembler take care of storage locations for the various names. All we have to do is to scan the symbol table, after generating assembly code for the program, and generate assembly language data definitions to be appended to the assembly language program for each name.

If machine code is to be generated by the compiler, however, then the position of each data object relative to a fixed origin, such as the beginning of an activation record must be ascertained. The same remark applies to a block of data loaded as a module separate from the program. For example, COMMON blocks in Fortran are loaded separately, and the positions of names relative to the beginning of the COMMON block in which they lie must be determined. For reasons discussed in Section 7.9, the approach of Section 7.3 has to be modified for Fortran, in that we must assign offsets for names after all declarations for a procedure have been seen and EQUIVALENCE statements have been processed.

In the case of names whose storage is allocated on a stack or heap, the compiler does not allocate storage at all — the compiler plans out the activation record for each procedure, as in Section 7.3.

The List Data Structure for Symbol Tables

The simplest and easiest to implement data structure for a symbol table is a linear list of records, shown in Fig. 7.33. We use a single array, or equivalently several arrays, to store names and their associated information. New names are added to the list in the order in which they are encountered. The position of the end of the array is marked by the pointer *available*, pointing to where the next symbol-table entry will go. The search for a name proceeds backwards from the end of the array to the beginning. When the name is located, the associated information can be found in the words following next. If we reach the beginning of the array without finding the name, a fault occurs — an expected name is not in the table.

Note that making an entry for a name and looking up the name in the symbol table are independent operations — we may wish to do one without the other. In a block-structured language, an occurrence of a name is in the scope of the most closely nested declaration of the name. We can implement this scope rule using the list data structure by making a fresh entry for a name every time it is declared. A new entry is made in the words immediately following the pointer *available*; that pointer is increased by the size of the symbol-table record. Since entries are inserted in order, starting from the beginning of the array, they appear in the order they are created in. By searching from *available* towards the beginning of the array, we are sure to find the most recently created entry.

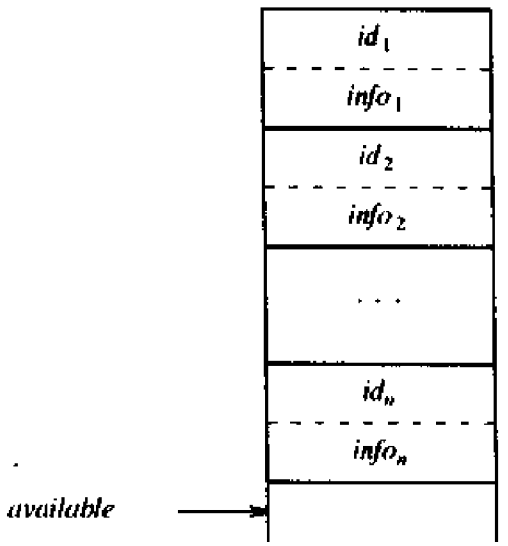


Fig. 7.33. A linear list of records.

If the symbol table contains n names, the work necessary to insert a new name is constant if we do the insertion without checking to see if the name is already in the table. If multiple entries for names are not allowed, then we need to look through the entire table before discovering that a name is not in the table, doing work proportional to n in the process. To find the data about a name, on the average, we search $n/2$ names, so the cost of an inquiry is also proportional to n. Thus, since insertions and inquiries take time proportional to n, the total work for inserting n names and making e inquiries is at most $cn(n+e)$, where c is a constant representing the time necessary for a few machine operations. In a medium-sized program, we might have $n = 100$ and $e = 1000$, so several hundred thousand machine operations are utilized in the bookkeeping. That may not be painful, since we are talking about less than a second of time. However, if n and e are multiplied by 10, the cost is multiplied by 100, and the bookkeeping time becomes prohibitive. Profiling yields valuable data about where a compiler spends its time and can be used to decide if too much time is being spent searching through linear lists.

Hash Tables

Variations of the searching technique known as hashing have been implemented in many compilers. Here we consider a rather simple variant known as *open hashing*, where "open" refers to the property that there need be no limit on the number of entries that can be made in the table. Even this scheme gives us the capability of performing e inquiries on n names in time proportional to $n(n+e)/m$, for any constant m of our choosing. Since m can be made as large as we like, up to n, this method is generally more efficient than linear lists and is the method of choice for symbol tables in most

situations. As might be expected, the space taken by the data structure grows with m, so a time-space tradeoff is involved.

The basic hashing scheme is illustrated in Fig. 7.34. There are two parts to the data structure:

1. A *hash table* consisting of a fixed array of m pointers to table entries.
2. Table entries organized into m separate linked lists, called *buckets* (some buckets may be empty). Each record in the symbol table appears on exactly one of these lists. Storage for the records may be drawn from an array of records, as discussed in the next section. Alternatively, the dynamic storage allocation facilities of the implementation language can be used to obtain space for the records, often at some loss of efficiency.

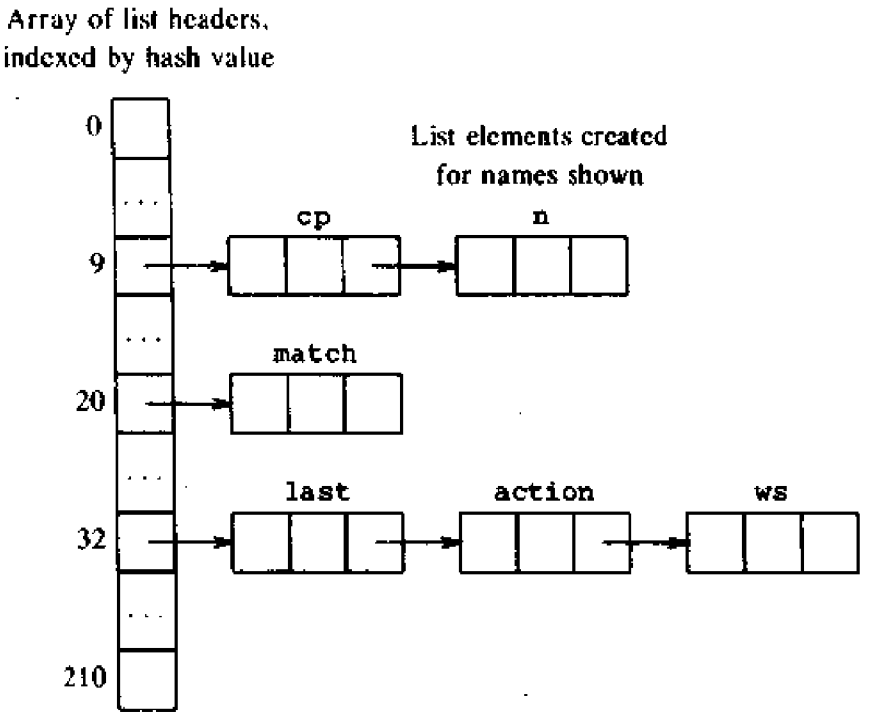


Fig. 7.34. A hash table of size 211.

To determine whether there is an entry for string s in the symbol table, we apply a *hash function* h to s, such that $h(s)$ returns an integer between 0 and $m-1$. If s is in the symbol table, then it is on the list numbered $h(s)$. If s is not yet in the symbol table, it is entered by creating a record for s that is linked at the front of the list numbered $h(s)$.

As a rule of thumb, the average list is n/m records long if there are n names in a table of size m. By choosing m so that n/m is bounded by a small constant, say 2, the time to access a table entry is essentially constant.

The space taken by the symbol table consists of m words for the hash table

and cn words for table entries, where c is the number of words per table entry. Thus the space for the hash table depends only on m, and the space for table entries depends only on the number of entries.

The choice of m depends on the intended application for a symbol table. Choosing m to be a few hundred should make table lookup a negligible fraction of the total time spent by a compiler, even for moderate-sized programs. When the input to a compiler might be generated by another program, however, the number of names can greatly exceed that of most human-generated programs of the same size, and larger table sizes might be preferable.

A great deal of attention has been given to the question of how to design a hash function that is easy to compute for strings of characters and distributes strings uniformly among the m lists.

One suitable approach for computing hash functions is to proceed as follows:

1. Determine a positive integer h from the characters $c_1, c_2, \ldots, c_k$ in string s. The conversion of single characters to integers is usually supported by the implementation language. Pascal provides a function *ord* for this purpose; C automatically converts a character to an integer if an arithmetic operation is performed on it.

2. Convert the integer h determined above into the number of a list, i.e., an integer between 0 and $m-1$. Simply dividing by m and taking the remainder is a reasonable policy. Taking the remainder seems to work better if m is a prime, hence the choice 211 rather than 200 in Fig. 7.34.

Hash functions that look at all characters in a string are less easily fooled than, say, functions that look only at a few characters at the ends or in the middle of a string. Remember, the input to a compiler may have been created by a program and may therefore have a stylized form chosen to avoid conflicts with names a person or some other program might use. People tend to "cluster" names as well, with choices like `baz`, `newbaz`, `baz1`, and so on.

A simple technique for computing h is to add up the integer values of the characters in a string. A better idea is to multiply the old value of h by a constant α before adding in the next character. That is, take $h_0 = 0$, $h_i = \alpha h_{i-1} + c_i$, for $1 \leq i \leq k$, and let $h = h_k$, where k is the length of the string. (Recall, the hash value giving the number of the list is h **mod** m.) Simply adding up the characters is the case $\alpha = 1$. A similar strategy is to exclusive-or c_i with αh_{i-1}, instead of adding.

For 32-bit integers, if we take $\alpha = 65599$, a prime near 2^{16}, then overflow soon occurs during the computation of αh_{i-1}. Since α is a prime, ignoring overflows and keeping only the lower-order 32 bits seems to do well.

In one set of experiments, the hash function *hashpjw* in Fig. 7.35 from P. J. Weinberger's C compiler did consistently well at all table sizes tested (see Fig. 7.36). The sizes included the first primes larger than 100, 200, . . . , 1500. A close second was the function that computed h by multiplying the old value by 65599, ignoring overflows, and adding in the next character. Function

```
(1)  #define PRIME 211
(2)  #define EOS '\0'
(3)  int hashpjw(s)
(4)  char   *s;
(5)  {
(6)      char *p;
(7)      unsigned h = 0, g;
(8)      for ( p = s; *p != EOS; p = p+1 ) {
(9)          h = (h << 4) + (*p);
(10)         if (g = h&0xf0000000) {
(11)             h = h ^ (g >> 24);
(12)             h = h ^ g;
(13)         }
(14)     }
(15)     return h % PRIME;
(16) }
```

Fig. 7.35. Hash function *hashpjw*, written in C.

hashpjw is computed by starting with $h = 0$. For each character c, shift the bits of h left 4 positions and add in c. If any of the four high-order bits of h is 1, shift the four bits right 24 positions, exclusive-or them into h, and reset to 0 any of the four high-order bits that was 1.

Example 7.10. For best results, the size of the hash table and the expected input must be taken into account when a hash function is designed. For example, it is desirable that the hash values for the most frequently occurring names in a language be distinct. If keywords are entered into the symbol table, then the keywords are likely to be among the most frequently occurring names, although in one sample of C programs, name `i` occurred over three times as often as `while`.

One way of testing a hash function is to look at the number of strings that fall onto the same list. Given a file F consisting of n strings, suppose b_j strings fall onto list j, for $0 \leq j \leq m-1$. A measure of how uniformly the strings are distributed across lists is obtained by computing

$$\sum_{j=0}^{m-1} b_j(b_j + 1)/2 \tag{7.2}$$

The intuitive justification for this term is that we need to look at 1 list element to find the first entry on list j, at 2 to find the second, and so on up to b_j to find the last entry. The sum of $1, 2, \ldots, b_j$ is $b_j(b_j+1)/2$.

From Exercise 7.14, the value of (7.2) for a hash function that distributes strings randomly across buckets is

$$(n/2m)(n + 2m - 1) \tag{7.3}$$

The ratio of the terms (7.2) and (7.3) is plotted in Fig. 7.36 for several hash functions applied to nine files. The files are:

1. The 50 most frequently occurring names and keywords in a sample of C programs.
2. Like (1), but with the 100 most frequently occurring names and keywords.
3. Like (1), but with the 500 most frequently occurring names and keywords.
4. 952 external names in the UNIX operating system kernel.
5. 627 names in a C program generated by C++ (Stroustrup [1986]).
6. 915 randomly generated character strings.
7. 614 words from Section 3.1 of this book.
8. 1201 words in English with xxx added as a prefix and suffix.
9. The 300 names `v100`, `v101`, . . . , `v399`.

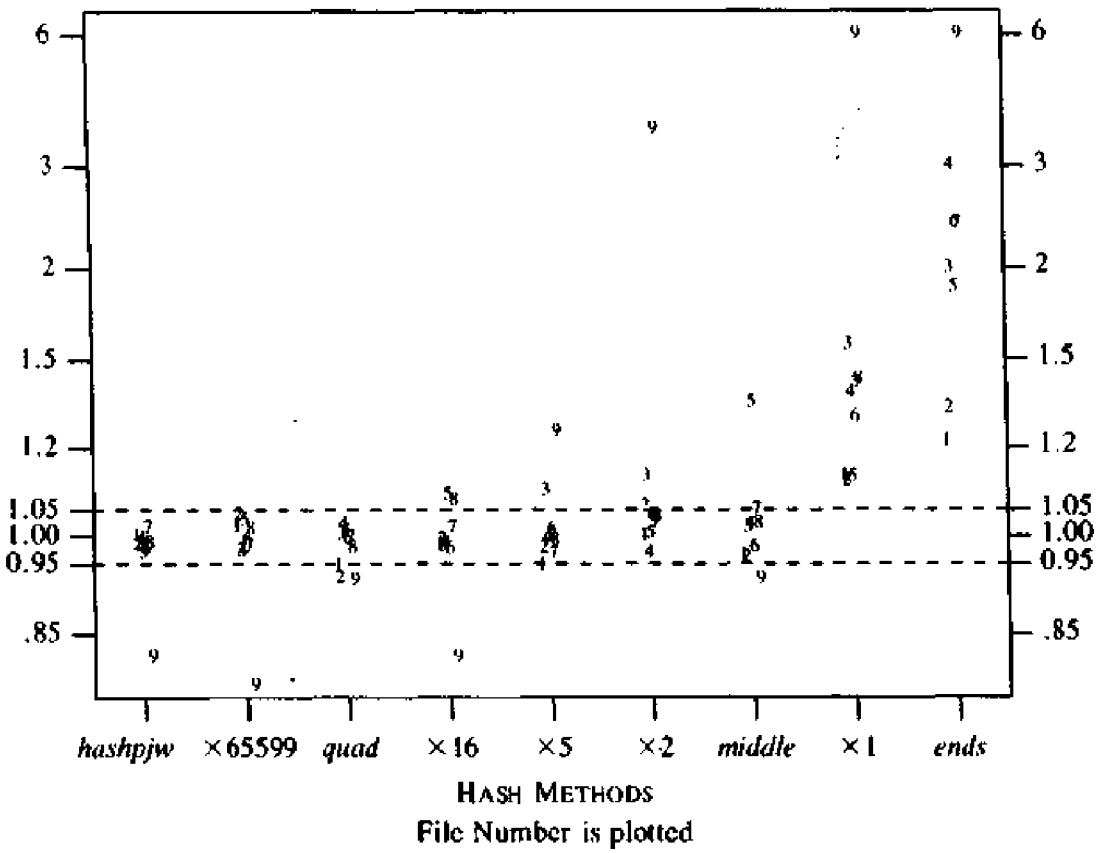


Fig. 7.36. Relative performance of hash functions for a table of size 211.

The function *hashpjw* is as in Fig. 7.35. The functions named $\times\alpha$, where α is an integer constant, compute h **mod** m, where h is obtained iteratively by starting with 0, multiplying the old value by α, and adding in the next

character. The function *middle* forms h from the middle four characters of a string, while *ends* adds up the first three and last three characters with the length to form h. Finally, *quad* groups every four consecutive characters into an integer and then adds up the integers. □

Representing Scope Information

The entries in the symbol table are for declarations of names. When an occurrence of a name in the source text is looked up in the symbol table, the entry for the appropriate declaration of that name must be returned. The scope rules of the source language determine which declaration is appropriate.

A simple approach is to maintain a separate symbol table for each scope. In effect, the symbol table for a procedure or scope is the compile-time equivalent of an activation record. Information for the nonlocals of a procedure is found by scanning the symbol tables for the enclosing procedures following the scope rules of the language. Equivalently, information about the locals of a procedure can be attached to the node for the procedure in a syntax tree for the program. With this approach the symbol table is integrated into the intermediate representation of the input.

Most closely nested scope rules can be implemented by adapting the data structures presented earlier in this section. We keep track of the local names of a procedure by giving each procedure a unique number. Blocks must also be numbered if the language is block-structured. The number of each procedure can be computed in a syntax-directed manner from semantic rules that recognize the beginning and end of each procedure. The procedure number is made a part of all locals declared in that procedure; the representation of the local name in the symbol table is a pair consisting of the name and the procedure number. (In some arrangements, such as those described below, the procedure number need not actually appear, as it can be deduced from the position of the record in the symbol table.)

When we look up a newly scanned name, a match occurs only if the characters of the name match an entry character for character, and the associated number in the symbol-table entry is the number of the procedure being processed. Most closely nested scope rules can be implemented in terms of the following operations on a name:

lookup:	find the most recently created entry
insert:	make a new entry
delete:	remove the most recently created entry

"Deleted" entries must be preserved; they are just removed from the active symbol table. In a one-pass compiler, information in the symbol table about a scope consisting of, say, a procedure body, is not needed at compile time after the procedure body is processed. However, it may be needed at run time, particularly if a run-time diagnostic system is implemented. In this case, the information in the symbol table must be added to the generated code for use

by the linker or by the run-time diagnostic system. See also the treatment of field names in records in Sections 8.2 and 8.3.

Each of the data structures discussed in this section — lists and hash tables — can be maintained so as to support the above operations.

When a linear list consisting of an array of records was described earlier in this section, we mentioned how *lookup* can be implemented by inserting entries at one end so that the order of the entries in the array is the same as the order of insertion of the entries. A scan starting from the end and proceeding towards the beginning of the array finds the most recently created entry for a name. The situation is similar in a linked list, as shown in Fig. 7.37. A pointer *front* points to the most recently created entry in the list. The implementation of *insert* takes constant time because a new entry is placed at the front of the list. The implementation of *lookup* is done by scanning the list starting at the entry pointed to by *front* and following links until the desired name is found, or the end of the list is reached. In Fig. 7.37, the entry for a declared in a block B_2, nested within block B_0, appears nearer the front of the list than the entry for a declared in B_0.

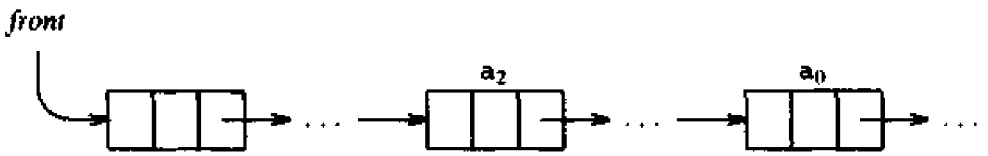


Fig. 7.37. The most recent entry for a is near the front.

For the *delete* operation, note that the entries for the declarations in the most deeply nested procedure appear nearest the front of the list. Thus, we do not need to keep the procedure number with every entry — if we keep track of the first entry for each procedure, then all entries up to the first can be deleted from the active symbol table when we finish processing the scope of this procedure.

A hash table consists of m lists accessed through an array. Since a name always hashes to the same list, individual lists are maintained as in Fig. 7.37. However, for implementing the *delete* operation we would rather not have to scan the entire hash table looking for lists containing entries to be deleted. The following approach can be used. Suppose each entry has two links:

1. a hash link that chains the entry to other entries whose names hash to the same value and
2. a scope link that chains all entries in the same scope.

If the scope link is left undisturbed when an entry is deleted from the hash table, then the chain formed by the scope links will constitute a separate (inactive) symbol table for the scope in question.

Deletion of entries from the hash table must be done with care, because deletion of an entry affects the previous one on its list. Recall that we delete the ith entry by making the $i-1$st entry point to the $i+1$st. Simply using the scope links to find the ith entry is therefore not enough. The $i-1$st entry can be found if the hash links form a circular linked list, in which the last entry points back to the first. Alternatively, we can use a stack to keep track of the lists containing entries to be deleted. A marker is placed in the stack when a new procedure is scanned. Above the marker are the numbers of the lists containing entries for names declared in this procedure. When we finish processing the procedure, the list numbers can be popped from the stack until the marker for the procedure is reached. Another scheme is discussed in Exercise 7.11.

7.7 LANGUAGE FACILITIES FOR DYNAMIC STORAGE ALLOCATION

In this section, we briefly describe facilities provided by some languages for the dynamic allocation of storage for data, under program control. Storage for such data is usually taken from a heap. Allocated data is often retained until it is explicitly deallocated. The allocation itself can be either *explicit* or *implicit*. In Pascal, for example, explicit allocation is performed using the standard procedure `new`. Execution of `new(p)` allocates storage for the type of object pointed to by `p` and `p` is left pointing to the newly allocated object. Deallocation is done by calling `dispose` in most implementations of Pascal.

Implicit allocation occurs when evaluation of an expression results in storage being obtained to hold the value of the expression. Lisp, for example, allocates a cell in a list when `cons` is used; cells that can no longer be reached are automatically reclaimed. Snobol allows the length of a string to vary at run time, and manages the space needed to hold the string in a heap.

Example 7.11. The Pascal program in Fig. 7.38 builds the linked list shown in Fig. 7.39 and prints the integers held in the cells; its output is

```
76      3
 4      2
 7      1
```

When execution of the program begins at line 15, storage for the pointer `head` is in the activation record for the complete program. Each time control reaches

```
(11)        new(p); p↑.key := k; p↑.info := i;
```

the call `new(p)` results in a cell being allocated somewhere within the heap; `p↑` refers to this cell in the assignments on line 11.

Note from the output of the program that the allocated cells are accessible when control returns to the main program from `insert`. In other words, cells allocated using `new` during an activation of `insert` are retained when control returns to the main program from the activation. □

```
(1)  program table(input, output);
(2)  type link = ↑ cell;
(3)       cell = record
(4)                   key, info : integer;
(5)                   next : link
(6)       end;
(7)  var  head : link;
(8)  procedure insert(k, i : integer);
(9)       var  p : link;
(10)      begin
(11)         new(p); p↑.key := k; p↑.info := i;
(12)         p↑.next := head; head := p
(13)      end;

(14) begin
(15)      head := nil;
(16)      insert(7,1); insert(4,2); insert(76,3);
(17)      writeln(head↑.key, head↑.info);
(18)      writeln(head↑.next↑.key, head↑.next↑.info);
(19)      writeln(head↑.next↑.next↑.key,
             head↑.next↑.next↑.info)
(20) end.
```

Fig. 7.38. Dynamic allocation of cells using new in Pascal.

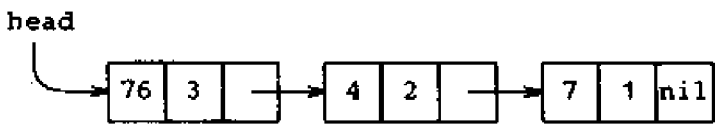


Fig. 7.39. Linked list built by program in Fig. 7.38.

Garbage

Dynamically allocated storage can become unreachable. Storage that a program allocates but cannot refer to is called *garbage*. In Fig. 7.38, suppose `nil` is assigned to `head↑.next` between lines 16 and 17:

```
(16)      insert(7,1); insert(4,2); insert(76,3);
          head↑.next := nil;
(17)      writeln(head↑.key, head↑.info);
```

The leftmost cell in Fig. 7.39 will now contain a `nil` pointer rather than a pointer to the middle cell. When the pointer to the middle cell is lost, the middle and rightmost cells become garbage.

Lisp performs *garbage collection*, a process discussed in the next section that reclaims inaccessible storage. Pascal and C do not have garbage collection, leaving it to the program explicitly to deallocate storage that is no longer desired. In these languages, deallocated storage can be reused, but garbage remains until the program finishes.

Dangling References

An additional complication can arise with explicit deallocation; dangling references can occur. As mentioned in Section 7.3, a dangling reference occurs when storage that has been deallocated is referred to. For example, consider the effect of executing `dispose(head↑.next)` between lines 16 and 17 in Fig. 7.38:

```
(16)    insert(7,1); insert(4,2); insert(76,3);
        dispose(head↑.next);
(17)    writeln(head↑.key, head↑.info);
```

The call to `dispose` deallocates the cell following the one pointed to by `head` as shown in Fig. 7.40. However, `head↑.next` has not been changed, so it is a dangling pointer to deallocated storage.

Dangling references and garbage are related concepts; dangling references occur if deallocation occurs before the last reference, whereas garbage exists if the last reference occurs before deallocation.

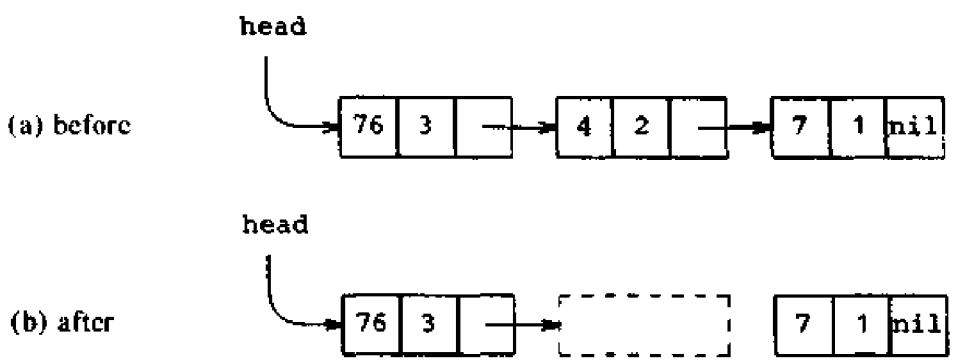


Fig. 7.40. Creation of dangling references and garbage.

7.8 DYNAMIC STORAGE ALLOCATION TECHNIQUES

The techniques needed to implement dynamic storage allocation depend on how storage is deallocated. If deallocation is implicit, then the run-time support package is responsible for determining when a storage block is no longer needed. There is less a compiler has to do if deallocation is done explicitly by the programmer. We consider explicit deallocation first.